

Développement et déploiement d'applications intelligentes

Support de cours et TP, 44 heures, 11 semaines

A. Larhlimi Élèves ingénieurs, année 2026-2027

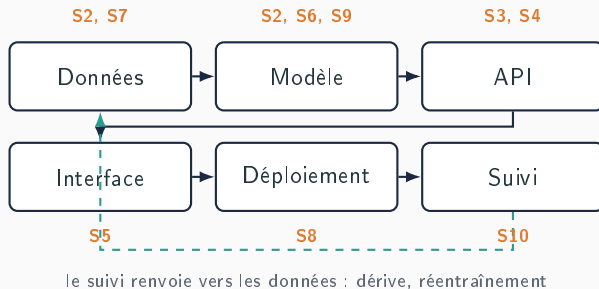


Objectif du cours : parcourir tout le cycle de vie

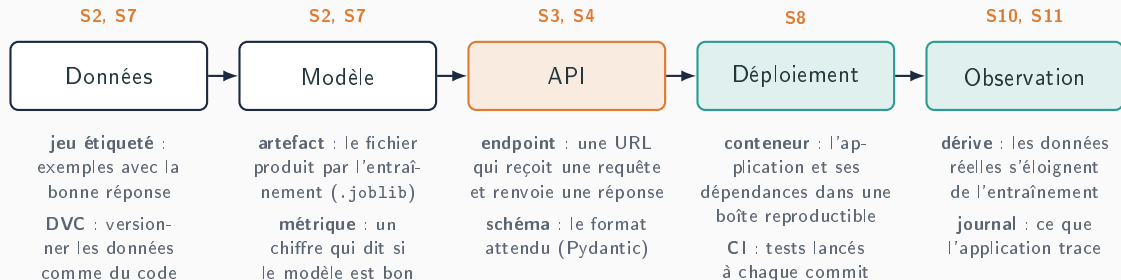
À la fin du cours, vous savez :

- cadrer un besoin, choisir l'approche ;
- entraîner un modèle reproductible ;
- l'exposer par une API testée ;
- construire une interface ;
- tracer expériences et données ;
- conteneuriser et mettre en ligne ;
- évaluer, sécuriser, documenter.

2 h cours + 2 h TP par semaine ; projet en groupe de 3 ou 4.



Le vocabulaire du cours, placé sur le cycle de vie



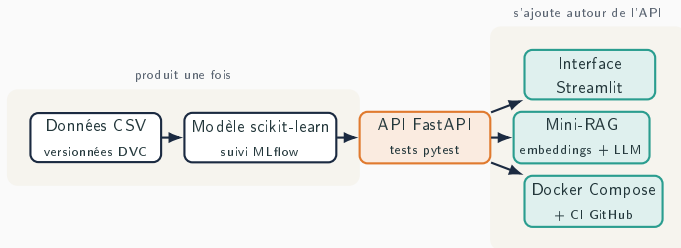
Dix mots suffisent pour suivre tout le cours. Chacun sera revu au moment où le fil rouge en a besoin; ce slide sert d'index à relire avant chaque TP.

Un seul fil rouge : l'analyse de sentiment d'avis clients

Pourquoi ce cas :

- données publiques en français (Allociné) ;
- modèle scikit-learn entraîné en une minute sur un portable ;
- extension naturelle vers le RAG en semaine 9.

La même application grandit de la semaine 1 à la semaine 11 : rien n'est jeté.

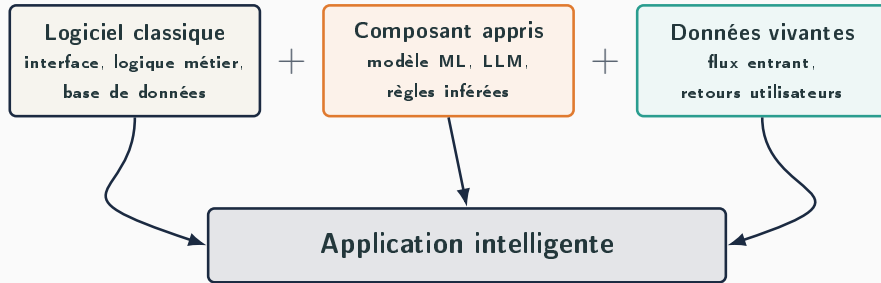


Pourquoi ce cours ? Du notebook à la production



*Ce cours ne porte pas sur l'entraînement des modèles :
il porte sur tout ce qui se trouve entre les deux.*

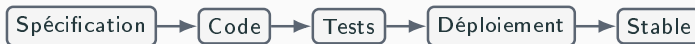
Qu'est-ce qu'une application intelligente ?



Exemples : détection de fraude, recommandation, maintenance prédictive, assistant de support, tarification dynamique.

Ce qui change par rapport au logiciel classique

Logiciel classique

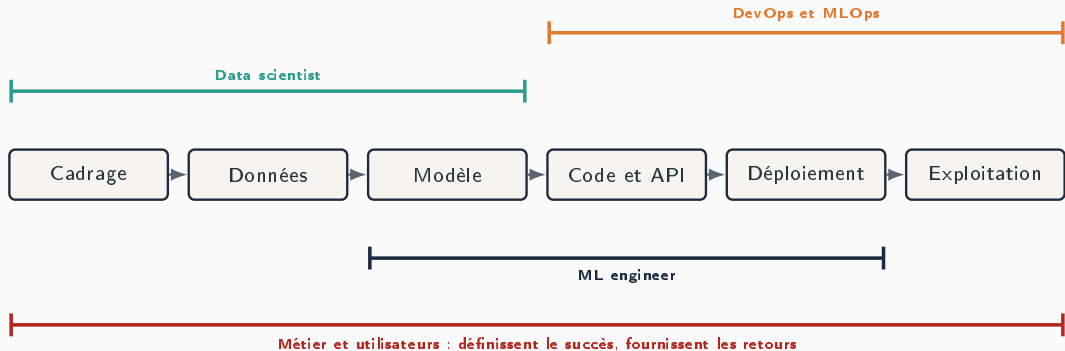


Application intelligente



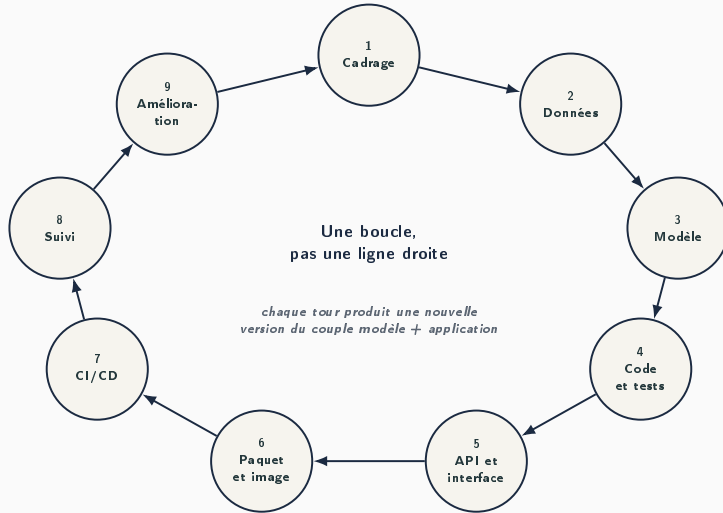
Trois artefacts à versionner (code, données, modèle), une sortie probabiliste, une performance qui se dégrade toute seule, une réussite qui se mesure en métrique ML et en métrique produit.

Qui fait quoi? Les rôles autour de l'application



*Dans une petite équipe, une seule personne porte les trois casquettes.
C'est le profil que ce cours prépare.*

Le cycle de vie d'une application intelligente



Les neuf étapes, en clair

1. Cadrage problème métier, utilisateurs, métriques de succès ; faut-il vraiment du ML ?

2. Données collecte, nettoyage, exploration, découpage entraînement / validation / test, versionnage

3. Modélisation baseline simple d'abord, entraînement, évaluation, suivi des expériences

4. Code et tests du notebook au projet Python modulaire, tests unitaires, linting, typage

5. API et interface exposition du modèle en REST (FastAPI), interface utilisateur (Streamlit)

6. Packaging paquet Python installable, image Docker reproductible

7. CI/CD intégration continue, tests automatiques à chaque commit, déploiement

8. Exploitation journalisation, latence, erreurs, dérive des données et des performances

9. Amélioration retours utilisateurs, réentraînement, nouvelle version, retour à l'étape 1

La chaîne d'outils du cours, volontairement minimale



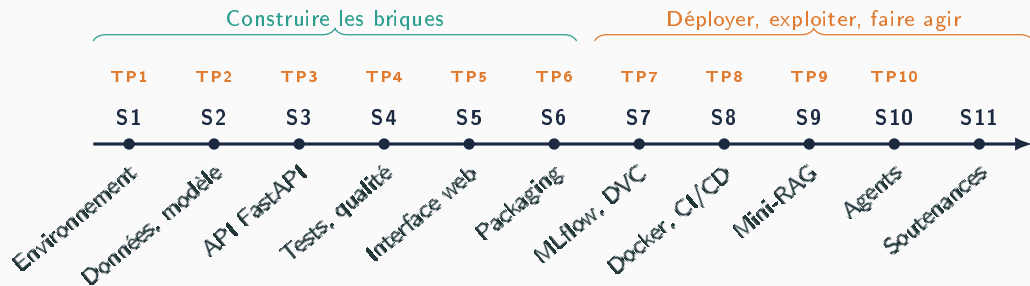
Git et GitHub : un dépôt par binôme, toute la trace du projet

Suivi des expériences et des données : MLflow et DVC (semaine 7)

Environnement : votre machine, Google Colab ou GitHub Codespaces

Une poignée d'outils, un dépôt, un environnement. Aucune plateforme payante, rien qui ne tourne pas sur un ordinateur portable.

Le cours en 11 semaines, une étape par semaine



*Un projet fil rouge en binôme traverse les onze semaines :
le TP de la semaine s'applique immédiatement à votre propre application.*



*Étude de
conception*

Soutenance

Ce qui est attendu

binôme constitué en semaine 1,
dépôt GitHub dès le TP1, un
commit par séance

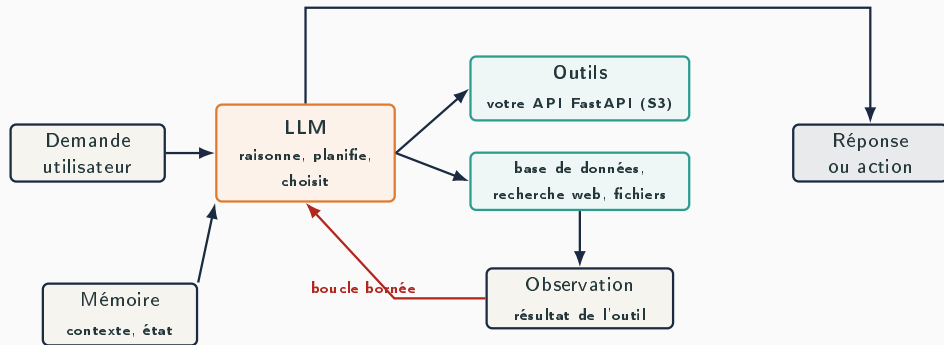
Ce qui est évalué

du code qui tourne chez quel-
qu'un d'autre que vous, des
choix justifiés, une application
accessible

Ce qui ne l'est pas

la performance brute du mo-
dèle : une baseline déployée
vaut mieux qu'un modèle par-
fait resté sur disque

Et l'IA agentique dans tout cela ? (semaine 10)

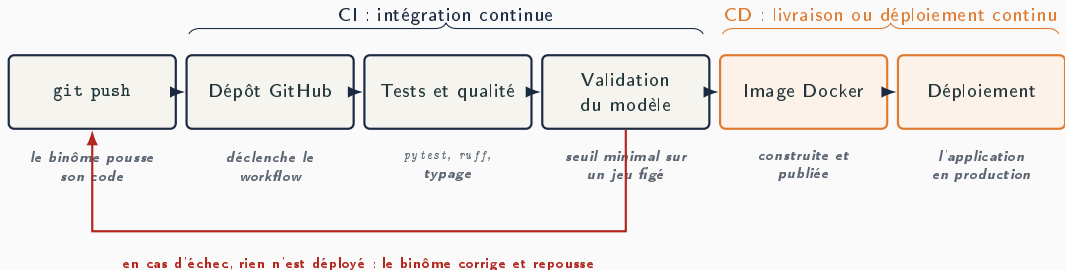


Un agent est une application intelligente, avec les mêmes exigences.

Les semaines 1 à 9 construisent ses briques (modèle, API, interface, paquet, déploiement, suivi) ; la semaine 10 les assemble en un système multi-agents autonome et coopératif. Ce qui change : sortie non déterministe, coût par appel, boucles à borner, évaluation sans exactitude simple, permissions des outils.

Une application intelligente prédit ; un agent agit. Ici, on déploie les deux.

CI/CD en une image : du commit à la production



Un seul fichier pilote tout cela : `.github/workflows/ci.yml` (semaine 8).

Différence avec un projet logiciel classique : la chaîne ne valide pas que le code, elle valide aussi le modèle (performance minimale, schéma des données d'entrée). Quand le réentraînement lui-même est automatisé, on parle de CT, *continuous training* : c'est l'étape 9 du cycle de vie rendue automatique.

Python ou Java ? Les outils équivalents, étape par étape

Étape	Python (le cours)		Java / JVM
S2 Données, modèle	pandas, scikit-learn	P	Tribo, Smile, DJL
S3 API REST	FastAPI	=	Spring Boot, Quarkus, Javalin
S4 Tests et qualité	pytest, ruff, mypy	J	JUnit 5, AssertJ, Checkstyle
S5 Interface	Streamlit	P	Vaadin, Thymeleaf, JS séparé
S6 Packaging	pip, wheel, pyproject	=	Maven ou Gradle, JAR, jlink
S7 MLflow, DVC	mlflow, dvc	=	client Java MLflow, DVC en CLI
S8 Docker, CI/CD	Dockerfile, Actions	=	Dockerfile ou Jib, Actions
S9 Mini-RAG	LangChain, Chroma	P	LangChain4j, pgvector
S10 Agents	SDK officiels	P	Spring AI
Suivi en production	logging, Prometheus	J	Actuator, Micrometer
Bac à sable	Google Colab	P	I Java, non officiel

P : avantage Python J : avantage Java = : équivalent. Six lignes sur onze sont neutres ou favorables à Java : le déploiement n'est pas le problème, l'apprentissage, l'interface et l'écosystème LLM le sont.

Basculer vers Java : le coût, le gain, la voie médiane

La voie médiane, celle de la pratique industrielle : entraîner en Python, servir en Java



Ce que la JVM apporte

- supervision native (Actuator, Micrometer)
- typage et outillage de test exigeants
- intégration au système d'information existant
- service robuste et performant en charge

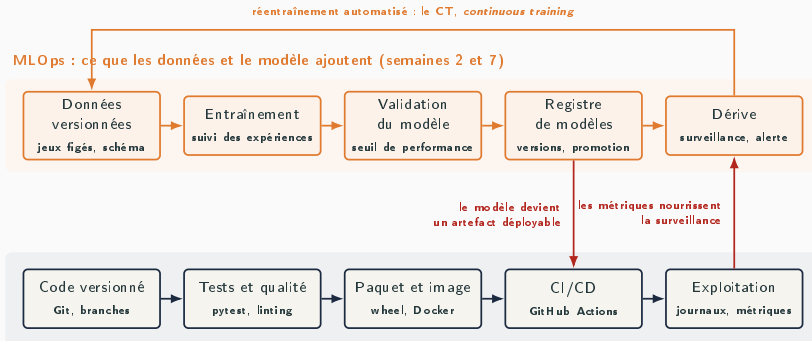
Ce qu'elle coûte

- écosystème d'apprentissage très en retrait
- aucun équivalent à Streamlit
- pas de Colab pour les étudiants
- verbosité qui mange les 44 heures

Le choix de la pile fait partie du travail demandé.

L'étude de conception (15 % de la note) demande de justifier Python, JVM ou l'architecture hybride ci-dessus, au regard du cas d'usage : latence attendue, système existant à intégrer, compétences de l'équipe, volume de données. Il n'y a pas de bonne réponse hors contexte.

DevOps, MLOps : où passe la frontière ?

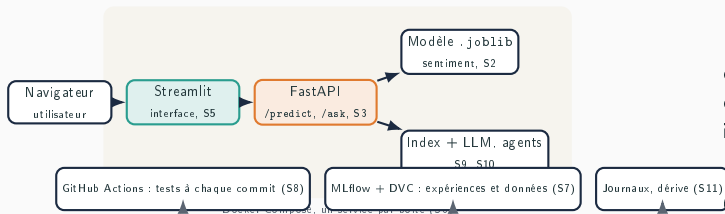


DevOps : le socle, valable pour n'importe quelle application (semaines 6 et 8)

Trois différences, et trois seulement.

1. Trois artefacts à versionner (code, données, modèle) au lieu d'un seul, et il faut pouvoir dire quelle version du modèle vient de quelles données avec quel code.
2. Une validation qui n'est pas binaire : un test passe ou échoue, un modèle a une performance qu'il faut comparer à un seuil.
3. Une dégradation sans modification : une application qu'on ne touche pas continue de fonctionner, un modèle qu'on ne touche pas se dégrade parce que le monde change.

Le fil rouge en semaine 11 : ce que vous aurez construit

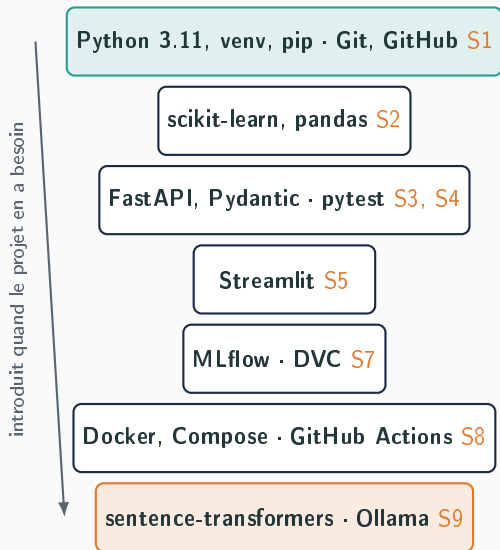


Une seule application, construite couche par couche : le modèle de la semaine 2 est encore celui qui répond en semaine 11, entouré d'une API, d'une interface, d'un index documentaire et d'une chaîne de livraison.

Ce que ce schéma dit du projet final : chaque boîte est notée séparément ; une boîte manquante n'empêche pas les autres de fonctionner.

Ce qu'il ne montre pas : la partie invisible (tests, documentation, décisions de cadrage), qui pèse la moitié de la note.

Neuf outils, pas quinze

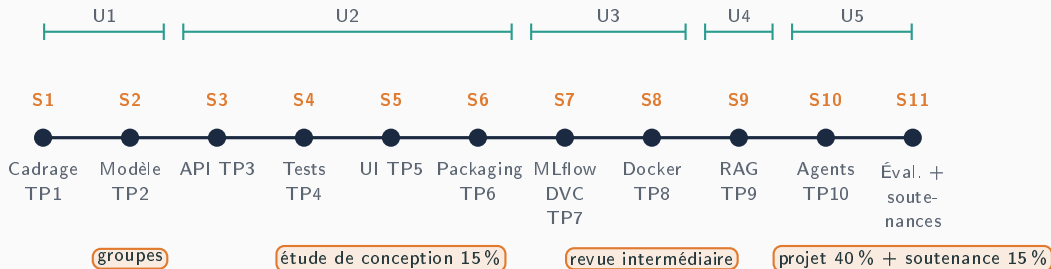


Chaque outil arrive quand le fil rouge l'exige, puis sert toutes les semaines suivantes.

Cité dans le syllabus	Traitement
Flask, Django, Express	comparés en 15 min (S3)
TensorFlow, PyTorch	présentés, non utilisés (S2)
Spring Boot	extension facultative
Kubeflow, Azure ML	étude documentaire (S11)
Kubernetes, AutoML	lecture conseillée, hors cours
LIME	remplacé par SHAP (S11)

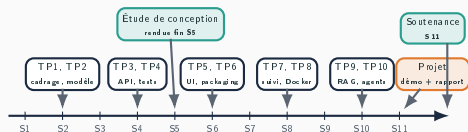
Tous gratuits, open source, sans GPU ni compte cloud payant.

Planning : 11 semaines, 10 TP, 1 projet



chaque semaine : 2 h de cours + 2 h de TP ; TP notés 30 % ; S11 : 1 h évaluation et sécurité, 3 h de soutenances

Évaluation : ce qui compte, et quand



Composante	Poids	Critère principal
10 TP (TP1 à TP10)	30 %	tests verts en CI, livrable conforme
Étude de conception (fin S5)	15 %	choix d'approche argumenté
Projet final (S10, S11)	40 %	application déployée, rapport de limites
Soutenance (S11)	15 %	démo en direct, réponses aux questions

Principe : on note ce qui fonctionne et ce qui est expliqué, pas la quantité de code. Trois tests rouges expliqués dans le README valent plus qu'un TP « vert » copié.

Chaque TP : un dépôt Git, une CI verte, un fichier docs/tpN.md d'une page (ce qui marche, ce qui ne marche pas, ce qui a été appris).

Projet final : même application que le fil rouge, sur un jeu de données choisi par le binôme, avec la démo en direct en S11.

Retard : un TP peut être rendu une semaine plus tard, une fois dans le semestre, sans pénalité.

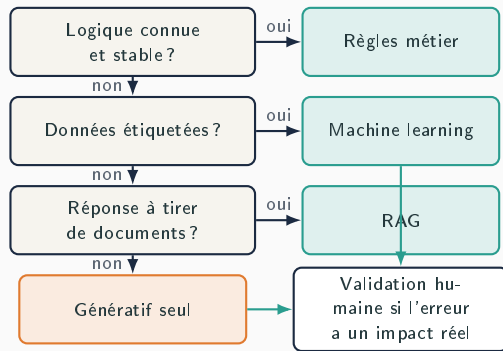
Répartition du syllabus du cours ; les modalités restent soumises au règlement de l'établissement.

Semaine 1 · Cours : choisir l'approche avant l'outil

Approche	Quand elle convient
Règles	logique connue, stable, explicable
ML classique	données étiquetées, tâche répétitive
RAG	répondre à partir de documents internes
Génératif seul	rédigier, résumer, reformuler

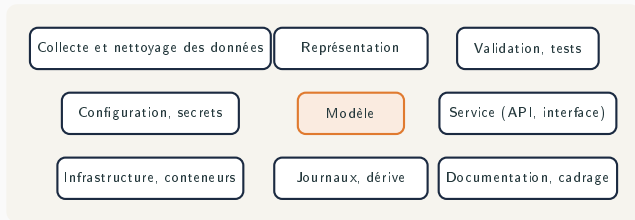
Chaque vague (règles 1980, statistique 1990, deep learning 2012, génératif 2018) s'ajoute à la précédente : une application moderne les combine.

Besoin exprimé : on descend tant que la réponse est « non »



Semaine 1 · Le modèle est la plus petite boîte de l'application

une application intelligente en production

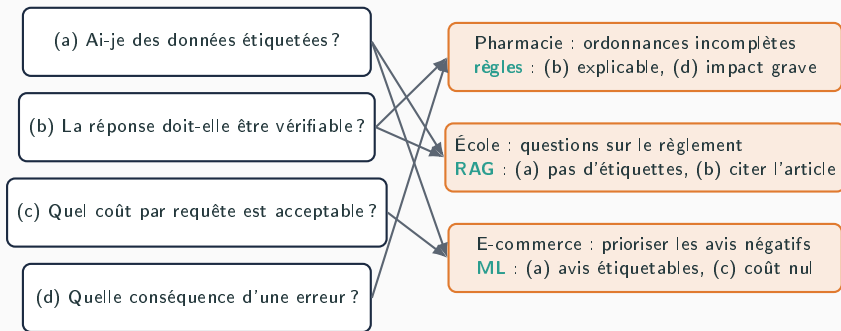


L'idée (Sculley et al., 2015, « Hidden technical debt in ML systems ») : dans un système réel, le code d'apprentissage est une fraction de l'ensemble. Tout le reste est du génie logiciel ordinaire, et c'est là que les projets échouent.

Conséquence pour le cours : sur 11 semaines, une seule (S2) est consacrée à l'entraînement. Les dix autres construisent les boîtes autour.

Conséquence pour vous : un bon modèle dans un notebook qu'on ne sait ni lancer, ni tester, ni expliquer n'est pas une application. C'est le critère de notation du projet final.

Semaine 1 · Cours : la grille de décision sur trois mini-cas



Règle du cours : partir de l'approche la plus simple qui satisfait les quatre questions, complexifier seulement si elle échoue. Deux réponses peuvent se défendre pour un même cas : c'est la justification qui est notée. Le cas 3 est le fil rouge du cours.

Semaine 1 · Chaque approche a un prix : quatre axes à comparer

	Règles	ML classique	RAG	Génératif seul
Données requises	aucune	milliers étiquetés	documents	aucune
Coût par requête	nul	négligeable	appel LLM	appel LLM
Latence	< 1 ms	quelques ms	secondes	secondes
Explicabilité	totale	poids lisibles	sources citées	faible

vert : favorable sable : intermédiaire orange : contrainte forte

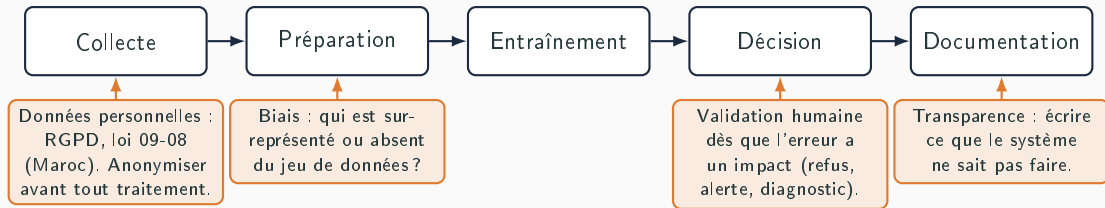
Lecture : aucune colonne n'est toute verte. Le choix est un compromis, à écrire noir sur blanc dans la fiche de cadrage du TP1.

Fil rouge : classer un avis en positif ou négatif se fait en quelques millisecondes avec un modèle de 40 Ko (ML classique). Demander la même chose à un LLM coûte mille fois plus et n'est pas plus juste sur cette tâche.

Quand le RAG devient le bon choix : la réponse doit venir de documents précis et être sourcée (S9). Le coût de l'appel est alors justifié par la valeur de la réponse.

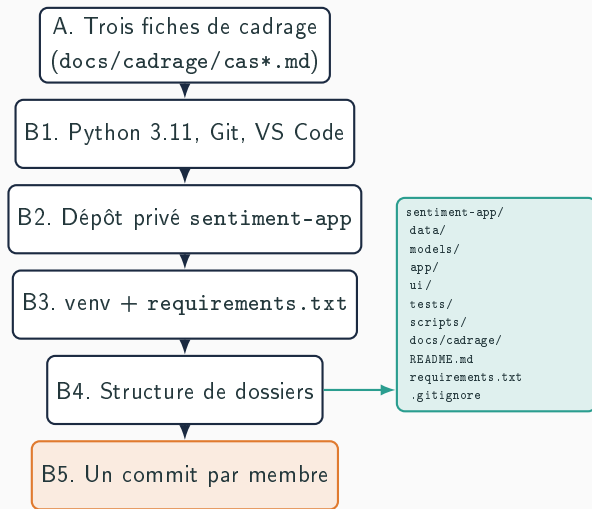
Exercice : ajouter une cinquième ligne « risque si le modèle se trompe » et la remplir pour les trois cas du TP1.

Semaine 1 · Cours : éthique et confidentialité, quatre points de contrôle



Règle du cours, du TP1 au projet final : uniquement des données publiques, fictives ou anonymisées.

Semaine 1 · TP1 : cadrer (50 min) puis outiller (70 min)

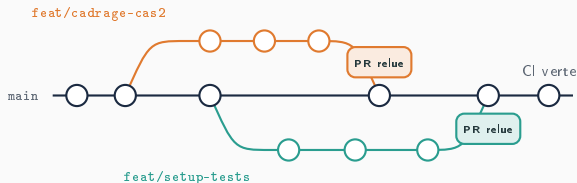


Chaque membre exécute, depuis son poste :

```
git clone <url-du-depot> && cd sentiment-app
python -m venv .venv
source .venv/bin/activate # Windows : .venv\Scripts\
                           activate
pip install -r requirements.txt
pip freeze > requirements.lock
python scripts/check_setup.py # 0 erreur attendue
pytest -q                     # 38 tests
```

Le kit du TP1 fournit scripts/setup.sh (et setup.ps1 sous Windows) qui enchaîne ces commandes. Piège classique : .venv/ commité, dépôt de 300 Mo ; le .gitignore fourni l'exclut.

Semaine 1 · Git à deux : une branche par tâche, une relecture par fusion



```
git switch -c feat/cadrage-cas2      # 1. une branche par tache
git add docs/cadrage/cas2.md && git commit -m "cadrage cas 2"
git push -u origin feat/cadrage-cas2 # 2. ouvrir la pull request
# 3. le binome relit, la CI tourne, on fusionne dans main
git switch main && git pull          # 4. repartir de main a jour
```

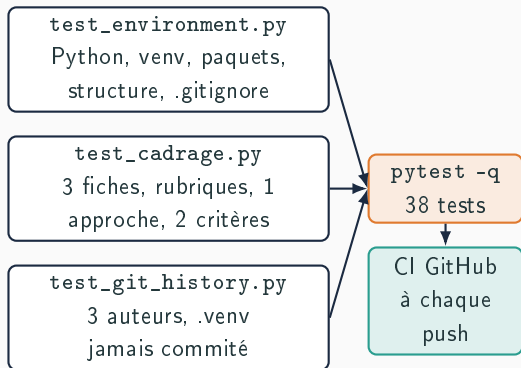
Règle du cours : personne ne commite directement sur main. Chaque tâche vit sur sa branche, passe par une pull request relue par l'autre membre, et n'est fusionnée que si la CI est verte (S8 automatisera ce contrôle).

Pourquoi : la relecture attrape les erreurs avant la fusion, et l'historique dit qui a fait quoi.

Conflit : deux branches modifient la même ligne. Git s'arrête et marque le fichier ; on choisit, on commite. Exercice du TP1 : provoquer un conflit volontaire sur README.md et le résoudre à deux.

Pièges : git add . qui embarque .env/ ; oublier git pull avant de créer la branche.

Semaine 1 · TP1 : la vérification est automatique dès le premier jour



scripts/check_setup.py refait les mêmes contrôles sous forme de diagnostic lisible (OK / ATTENTION / ERREUR).

Extrait de test_cadrage.py :

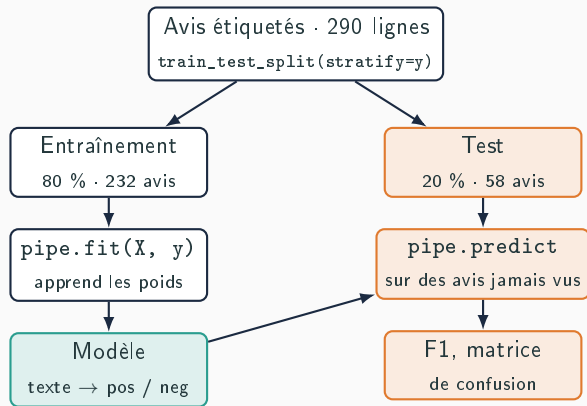
```
CRITERIA = ["(a)", "(b)", "(c)", "(d)"]

def test_exactly_one_approach_checked(cadrage_dir):
    for f in cadrage_dir.glob("cas*.md"):
        text = f.read_text(encoding="utf-8")
        assert text.count("- [x]") == 1, f.name

def test_justification_cites_two_criteria(cadrage_dir):
    for f in cadrage_dir.glob("cas*.md"):
        body = section_body(f.read_text(), "Justification")
        assert sum(c in body for c in CRITERIA) >= 2
```

Testé sur un dépôt simulé : **3 échecs** tant que les fiches sont vides, **38 réussites** une fois remplies. TP validé quand la CI est verte.

Semaine 2 · Apprentissage supervisé : apprendre sur un jeu, juger sur un autre



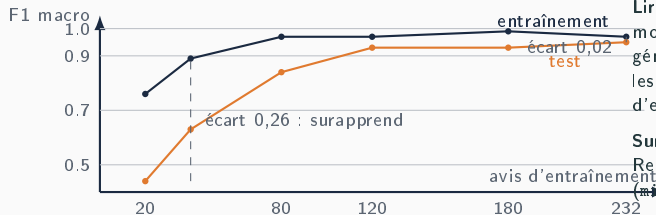
Supervisé : on fournit des exemples avec la bonne réponse ; le modèle ajuste ses paramètres pour reproduire ces réponses.

Pourquoi séparer : un modèle jugé sur ses propres exemples d'entraînement paraît toujours bon. Le jeu de test simule les avis que l'application recevra en production.

Stratifier : garder la même proportion pos / neg dans les deux jeux, sinon les métriques mentent.

Fil rouge : avis Allociné, deux classes ; l'extrait de démonstration (290 avis) sert aux tests, le vrai jeu en séance.

Semaine 2 · Sur- et sous-apprentissage : lire la courbe d'apprentissage



Mesuré par `experiences.py` sur le jeu de démonstration, même jeu de test (58 avis) à chaque point.

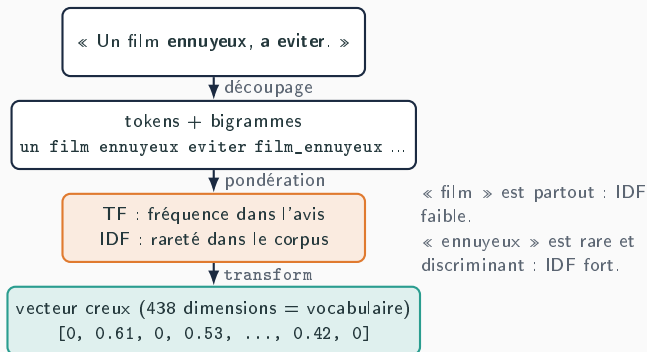
Lire les deux courbes ensemble. Avec 20 avis, le modèle « connaît » ses exemples (0,76) mais ne généralise pas (0,44) : il a appris par cœur. À 232 avis, les deux courbes se rejoignent : ce qu'il sait sur le jeu d'entraînement, il le sait aussi sur des avis nouveaux.

Surapprentissage : écart large entre les courbes.
Remèdes : plus de données, moins de paramètres (moins de df, unigrammes), régularisation.

Sous-apprentissage : courbes basses et proches.
Remèdes : représentation plus riche (bigrammes, embeddings en S9), modèle plus souple.

Fil rouge : le plateau arrive vers 120 avis ; 100 de plus n'apportent que 0,02. Tracer la même courbe sur Allociné avant de collecter davantage.

Semaine 2 · Représenter du texte : du sac de mots à TF-IDF



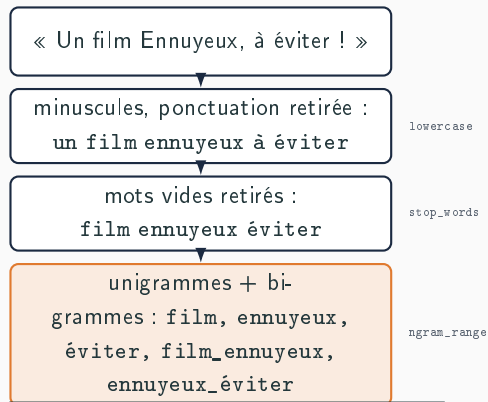
```
vec = TfidfVectorizer(ngram_range=(1, 2))
X = vec.fit_transform(df["avis"])
X.shape # (290, 438), creux
```

Limites : pas de sens, pas d'ordre au-delà des bigrammes; « pas mauvais » et « mauvais » se ressemblent.

Pourquoi c'est suffisant ici : le sentiment tient à quelques mots forts; TF-IDF les fait ressortir, s'entraîne en une seconde et se déploie sans GPU.

Suite : les embeddings (sentence-transformers) en semaine 9, quand le sens comptera.

Semaine 2 · Prétraiter le texte : mesurer avant de croire



Variante (experiences.py)	Vocabulaire	F1 test
référence, bigrammes, minuscules	411	0,948
unigrammes seuls	92	0,948
mots vides retirés (16 mots)	381	0,948
min_df=2	281	0,948
sans mise en minuscules	424	0,948

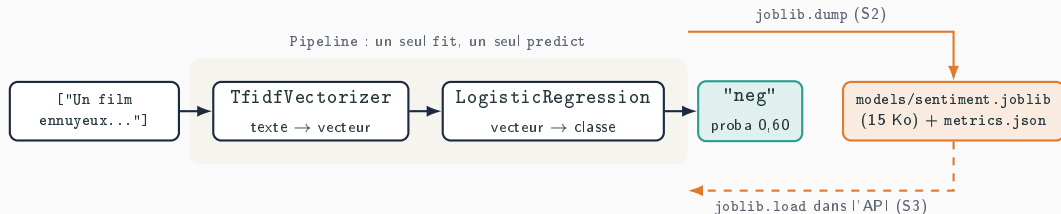
Le résultat surprend, et c'est le point du slide : sur le jeu de démonstration, aucun prétraitement ne change le F1. Le vocabulaire fond de 411 à 92 sans perte : les mots qui décident sont peu nombreux et déjà présents.

Pourquoi mesurer quand même : sur Allociné (vocabulaire de plusieurs dizaines de milliers de mots, fautes, argot), les mêmes options font bouger le F1 de un à trois points et divisent la taille du modèle. À vérifier à l'étape D du TP2.

En interne : TfidfVectorizer met en minuscules et découpe sur les non-lettres ; il ne retire ni accents ni mots vides français par défaut.

Règle : un prétraitement se justifie par une mesure, pas par une habitude.

Semaine 2 · Le Pipeline : un seul objet à entraîner, sauvegarder, déployer

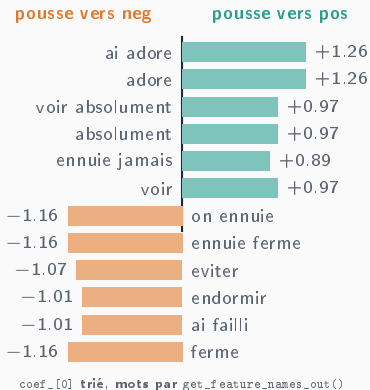


```
pipe = Pipeline([
    ("tfidf", TfidfVectorizer(ngram_range=(1, 2))),
    ("clf", LogisticRegression(max_iter=1000))]
)
pipe.fit(X_tr, y_tr)
joblib.dump(pipe, "models/sentiment.joblib")
# plus tard, dans l'API (S3) :
pipe = joblib.load("models/sentiment.joblib")
pipe.predict_proba(["Un film magnifique"])
```

Pourquoi un Pipeline : le vectoriseur est ajusté sur le jeu d'entraînement et réutilisé tel quel. Sans Pipeline, on refait `fit_transform` en production et le vocabulaire change : les prédictions deviennent fausses sans erreur visible.

Règle du cours : le `.joblib` est le seul artefact qui passe de S2 aux semaines suivantes ; suivi par MLflow (S7), jamais commité dans Git.

Semaine 2 · Ouvrir le modèle : les poids de la régression logistique se lisent



```
mots = pipe["tfidf"].get_feature_names_out()
coef = pipe["clf"].coef_[0] # un poids par mot
ordre = coef.argsort()
print(mots[ordre[:6]]) # les plus negatifs
print(mots[ordre[-6:]]) # les plus positifs
```

Comment lire : la régression logistique calcule une somme pondérée des mots présents, puis la transforme en probabilité. Un poids positif pousse vers « pos », un poids négatif vers « neg ». « adoré » et « à éviter » sont les mots les plus décisifs du modèle réel.

Pourquoi c'est précieux : on peut expliquer une prédiction à un utilisateur (« négatif à cause de *ennuyeux* et *éviter* »), détecter un mot parasite (un nom de film appris comme signal) et documenter les limites (S11).

Exercice : afficher les 20 poids les plus forts sur Allociné et repérer un mot qui n'a rien à voir avec le sentiment.

Semaine 2 · Évaluer : accuracy ne suffit pas, lire la matrice de confusion

réel \ prédit	pos	neg
pos	29 vrais positifs	0 faux négatifs
neg	3 faux positifs	26 vrais négatifs

Sortie réelle de `python evaluate.py` sur les 58 avis de test

Métrique	Formule (classe pos)	Obtenu
Accuracy	$(29+26)/58$	0,948
Précision	$29/(29+3)$	0,906
Rappel	$29/(29+0)$	1,000
F1 macro	moyenne des F1 des deux classes	0,948

Lire la matrice : les 3 erreurs sont des avis négatifs pris pour positifs. Ce sont les avis mitigés (« brillant au début, mais finalement une déception ») : le sac de mots voit « brillant » et ignore le « mais ».

Précision ou rappel ? Dépend du produit. Alerter le service client sur les avis négatifs : on veut du rappel (ne rater aucun mécontent). Publier automatiquement les avis positifs : on veut de la précision.

F1 macro : moyenne des F1 par classe, robuste au déséquilibre ; métrique suivie dans MLflow (S7) et seuil testé par pytest ($\geq 0,80$).

Piège : sur un jeu 95 % positif, prédire toujours « pos » donne 95 % d'accuracy et un F1 macro de 0,49.

Semaine 2 · Un score unique est un tirage, la validation croisée une fourchette

p	test	entraîn.	entraîn.	entraîn.	entraîn.	F1 = 0.965
p	entraîn.	test	entraîn.	entraîn.	entraîn.	F1 = 0.931
p	entraîn.	entraîn.	test	entraîn.	entraîn.	F1 = 0.931
p	entraîn.	entraîn.	entraîn.	test	entraîn.	F1 = 0.878
p	entraîn.	entraîn.	entraîn.	entraîn.	test	F1 = 0.948

290 avis mélangés puis découpés en 5 parts égales
et stratifiées; chaque part sert une fois de test

moyenne 0,931 écart-type 0,029 soit F1 entre 0,90 et 0,96

```
from sklearn.model_selection import (  
    cross_val_score, StratifiedKFold)  
cv = StratifiedKFold(5, shuffle=True, random_state=42)  
scores = cross_val_score(pipe, df["avis"], df["note"],  
                          cv=cv, scoring="f1_macro")  
scores.mean(), scores.std()    # 0.931, 0.029
```

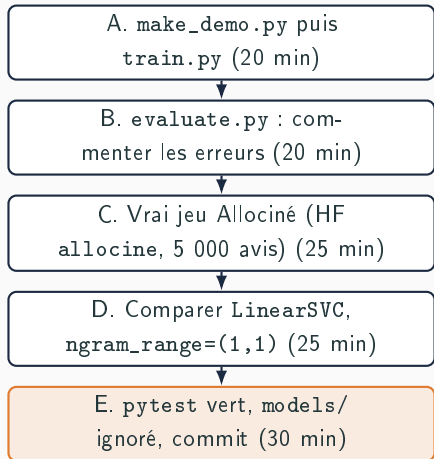
Le 0,948 du slide précédent vient d'un seul découpage. Cinq découpages donnent de 0,878 à 0,965 : la « vraie » valeur est une fourchette, pas un chiffre.

Conséquence pour l'étape D du TP2 : deux variantes dont les F1 diffèrent de moins d'un écart-type (ici 0,03) sont indiscernables. Comparer des moyennes de validation croisée, pas des scores uniques.

Quand s'en passer : jeu très grand (Allociné complet, 160 000 avis) : un découpage suffit.

Piège : faire la validation croisée sur le jeu de test. Elle se fait sur le jeu d'entraînement ; le test reste intouché jusqu'à la fin.

Semaine 2 · TP2 : entraîner, évaluer, sauvegarder, tester (2 h)



Kit : code/semaine2/ (train.py, evaluate.py, predict.py, tests/) et TP2-colab.ipynb.

Les 4 tests : fichiers produits, F1 macro $\geq 0,80$, deux prédictions de bon sens, predict.py renvoie un label et une confiance entre 0,5 et 1.

Livrable : models/metrics.json du vrai jeu, tableau de l'étape D dans docs/tp2.md, tests verts.

Pièges : évaluer sur le jeu d'entraînement (F1 de 1,0 suspect); oublier stratify; commiter le .joblib.

```
$ python train.py
F1 macro (test) : 0.948
Modele sauvegarde : models/sentiment.joblib
$ python -m pytest -q
.... [100%] 4 passed in 1.98s
```

Ce qui attend en S3 : predict.py devient la fonction appelée par la route POST /predict de l'API FastAPI.

Semaine 3 · Pourquoi une API : le modèle derrière une adresse



Le problème : après le TP2, le modèle est un fichier sur votre disque. Le collègue qui veut l'utiliser doit installer Python, scikit-learn, la bonne version, et copier le fichier. À la prochaine version, tout le monde recommence.

La réponse : une API, c'est un programme qui écoute une adresse réseau, reçoit du texte au format JSON et renvoie la prédiction au même format. N'importe quel langage sait faire ça.

Ce qui change pour le cours : à partir de cette semaine, tout ce que vous construisez (interface, RAG, monitoring) parle à l'API, jamais au fichier `.joblib`.

Vocabulaire : *client* (qui appelle), *serveur* (qui répond), *endpoint* (une URL qui fait une chose).

Semaine 3 · HTTP en cinq minutes : une requête, une réponse, un code



```
$ curl -X POST localhost:8000/predict \
  -H "Content-Type: application/json" \
  -d '{"texte": "Un film ennuyeux, a éviter."}'
{"label": "neg", "confiance": 0.636,
"version_modele": "tfidf-logreg-f1=0.948"}
```

JSON : du texte structuré (objets {}, listes [], chaînes, nombres) que tous les langages lisent. C'est le format d'échange de l'API; un DataFrame ou un objet Python ne passe pas sur le réseau.

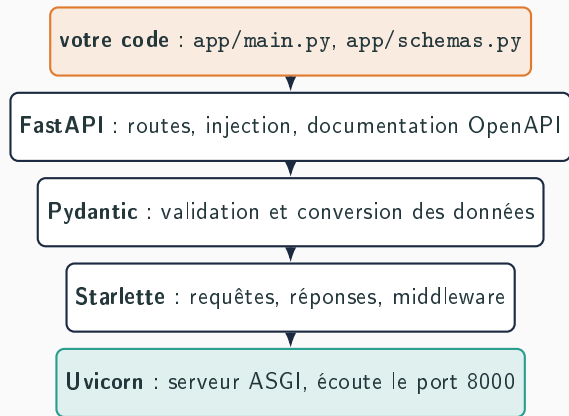
Sans état : chaque requête est indépendante ; le serveur n'a aucun souvenir de la précédente. Ce qui doit être gardé (le modèle) est chargé au démarrage (slide 21).

Règle : le code de statut dit à qui revient l'erreur. Une API qui renvoie 200 avec « erreur » dans le corps ment à son client.

Méthode	Sens	Dans le fil rouge
GET	lire, sans effet	/health, /docs
POST	envoyer des données	/predict
PUT, DELETE	modifier, supprimer	pas cette semaine

Code	Sens	Qui a tort
200	réussi	personne
404	route inconnue	le client (URL)
422	données invalides	le client (contenu)
500	erreur interne	le serveur
503	service indisponible	le serveur (modèle absent)

Semaine 3 · L'outil : FastAPI, et la pile sur laquelle il repose



```
$ pip install "fastapi[standard]"
$ uvicorn app.main:app --reload
INFO:      Application startup complete.
INFO:      Uvicorn running on http://127.0.0.1:8000
```

FastAPI (2018, Sebastián Ramírez) : un cadre Python pour écrire des API. On déclare une fonction, on l'annote avec une route, et le cadre s'occupe de lire la requête, valider les données, appeler la fonction et sérialiser la réponse.

Trois raisons de le choisir pour ce cours :

- les annotations de type Python servent de contrat : pas de code de validation à écrire ;
- la documentation interactive (`/docs`) est générée sans effort ;
- `TestClient` permet de tester l'API dans `pytest` sans lancer de serveur.

ASGI : le standard qui relie un serveur (Uvicorn) à une application Python ; `app` dans `app.main:app` est l'objet que le serveur appelle à chaque requête.

`--reload` : redémarre à chaque sauvegarde, pour le développement seulement.

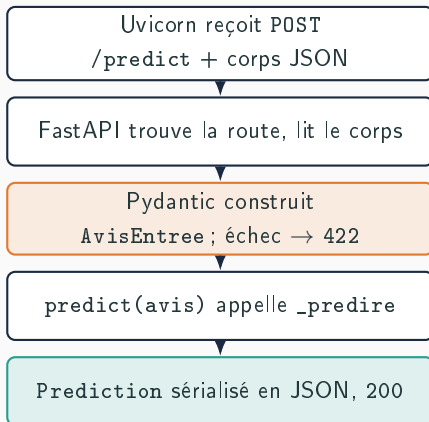
Semaine 3 · Première route : le trajet d'une requête dans app/main.py

```
app = FastAPI(title="Sentiment API", version="0.1.0",
              lifespan=lifespan)

@app.get("/health") # 1
def health():
    return {"status": "ok", "modele_charge": "pipe" in etat}

@app.post("/predict", response_model=Prediction) # 2
def predict(avis: AvisEntree): # 3
    if "pipe" not in etat:
        raise HTTPException(503, "Modele non charge") # 4
    return _predire([avis.texte])[0] # 5
```

1. Route de vie : le déploiement (S8) l'interroge.
2. POST + response_model : la réponse est filtrée et documentée.
3. Le paramètre typé AvisEntree : FastAPI lit et valide le corps JSON.
4. Erreur explicite avec le bon code, jamais un 200 vide.
5. Fonction métier séparée de la route : testable sans HTTP.



Mesuré avec TestClient, 200 requêtes : médiane 1,4 ms, p95 1,9 ms par prédiction. Le réseau coûtera plus que le modèle.

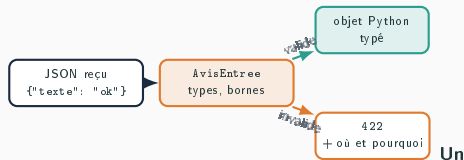
Semaine 3 · Pydantic : le contrat de l'API, et ce qui arrive quand on le viole

```
# app/schemas.py
class AvisEntree(BaseModel):
    texte: str = Field(min_length=3, max_length=2000,
                       examples=["Un film magnifique."])

class Prediction(BaseModel):
    label: str          # "pos" ou "neg"
    confiance: float     # entre 0.5 et 1
    version_modele: str

class LotEntree(BaseModel):
    textes: list[str] = Field(min_length=1, max_length=100)
```

```
$ curl -X POST ../predict -d '{"texte":"ok"}'      # trop court
422 {"detail":[{"type":"string_too_short",
  "loc":["body","texte"],
  "msg":"String should have at least 3 characters",
  "input":"ok","ctx":{"min_length":3}}]}
```



schéma, trois usages : validation à l'entrée, filtrage à la sortie (response_model retire tout champ non déclaré), documentation (/docs affiche l'exemple).

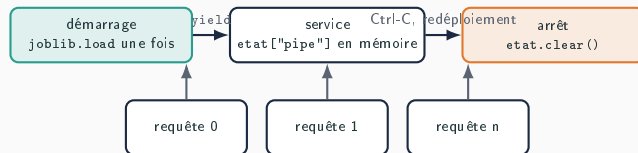
Le 422 est une bonne nouvelle : l'erreur est attrapée avant le modèle, avec le chemin du champ fautif. Sans schéma, on aurait un 500 obscur dans scikit-learn.

Bornes utiles : max_length protège contre un texte de 10 Mo ; max_length=100 sur le lot borne le temps d'une requête.

Exercice : ajouter langue: str = "fr" avec un

Literal["fr", "en"] et observer le 422.

Semaine 3 - Charger le modèle une fois : le cycle de vie de l'application



toutes lisent le même objet ; aucune ne recharge le fichier

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    etat["pipe"] = joblib.load(MODELE)      # avant la 1re requete
    etat["version"] = "tfidf-logreg-f1=0.948"
    yield                                  # l'application tourne
    etat.clear()                           # a l'arrêt

app = FastAPI(lifespan=lifespan)
```

L'erreur classique : `joblib.load` dans la fonction `predict`. Le fichier est relu à chaque requête. Mesuré ici : 0,9 ms pour un modèle de 15 Ko, donc invisible sur le jeu de démonstration ; sur un modèle Allociné de plusieurs Mo, c'est 50 à 500 ms par appel, et le disque devient le goulot.

lifespan : une fonction qui s'exécute avant la première requête (partie avant `yield`) et après la dernière (partie après). Le modèle vit dans un dictionnaire `etat` partagé par toutes les requêtes.

Pourquoi la version dans la réponse : quand deux modèles coexistent (S7, MLflow), le client sait lequel a répondu.

Chemin par variable d'environnement (MODELE) : le même code sert en test, sur votre poste et dans Docker (S8) sans modification.

Semaine 3 · Tester l'API sans serveur : TestClient dans pytest

```
@pytest.fixture(scope="module")
def client():
    os.environ["MODELE"] = "models/sentiment.joblib"
    from app.main import app
    with TestClient(app) as c:      # declenche lifespan
        yield c

def test_predict_positif(client):
    r = client.post("/predict",
                    json={"texte": "Un film magnifique, j'ai
                           adore."})
    assert r.status_code == 200
    assert r.json()["label"] == "pos"

def test_validation_422(client):
    r = client.post("/predict", json={"texte": "ok"})
    assert r.status_code == 422
    assert r.json()["detail"][0]["loc"] == ["body", "texte"]
```

```
$ python -m pytest -q
.....
7 passed in 3.18s [100%]
```

/health → 200, « modèle chargé »

avis positif, avis négatif → bon
label, confiance dans [0,5 ; 1]

texte trop court → 422, champ texte désigné

champ absent (avis au lieu de texte) → 422

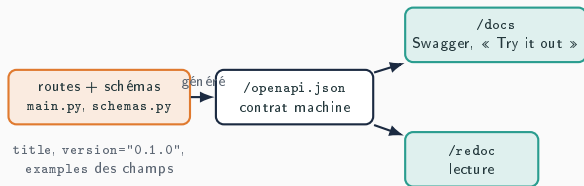
lot de 2 avis → 2 prédictions, dans l'ordre

route inconnue → 404

TestClient appelle l'application en mémoire, sans port ni réseau : les tests tournent en 3 s et passent en CI (S8) sans rien installer de plus.

On teste autant les refus (422, 404) que les succès : une API se juge à la façon dont elle dit non.

Semaine 3 · Documentation automatique, version, erreurs : le contrat visible



Situation	Code	Comment on le produit
corps JSON invalide	422	Pydantic, automatique
URL inconnue	404	FastAPI, automatique
modèle non chargé	503	raise HTTPException(503, ...)
exception non prévue	500	FastAPI; à journaliser (S10)

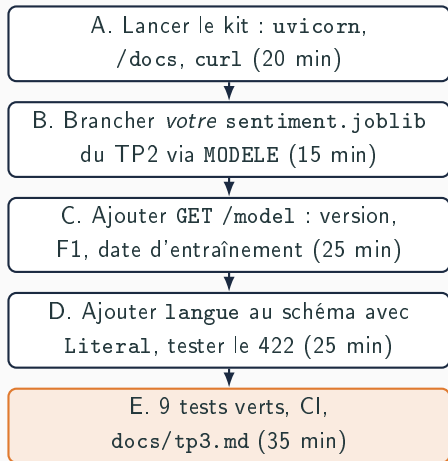
Vérifié sur l'API du cours : GET /docs répond 200 ; /openapi.json expose exactement trois chemins, /health, /predict, /predict/batch. La documentation ne peut pas diverger du code : elle en est extraite.

Version : version="0.1.0" sur l'application, version_model dans chaque réponse. La première dit quel code tourne, la seconde quel modèle a répondu. Les deux servent au débogage en production.

Règle des erreurs : anticiper ce qui peut manquer (modèle, champ, format) et répondre avec le code juste ; laisser remonter le reste en 500 plutôt que d'attraper toutes les exceptions et de masquer les bugs.

Exercice : ouvrir /docs, exécuter /predict depuis le navigateur, puis provoquer un 422 et lire la réponse.

Semaine 3 · TP3 : exposer le modèle du TP2 par une API testée (2 h)



Kit :

code/semaine3/ (app/main.py, app/schemas.py,
tests/test_api.py, requirements.txt) et TP3-colab.ipynb.

Vous partez de l'API du cours (7 tests verts) et du modèle entraîné au TP2. Le kit reconstruit un modèle de démonstration si le vôtre manque.

Vous rendez : le dépôt avec les deux routes ajoutées, deux nouveaux tests (/model et le 422 sur langue), la CI verte, et docs/tp3.md avec la copie d'une réponse 200 et d'une réponse 422 commentées.

Grille (sur 10) : tests verts en CI (4); /model renvoie des valeurs lues dans metrics.json, pas codées en dur (2); 422 correct sur langue (2); docs/tp3.md (2).

Pièges : joblib.load dans la route; oublier with TestClient(app) (le lifespan ne s'exécute pas); chemin du modèle en dur.

Colab : le serveur ne s'expose pas, le notebook utilise TestClient pour tout exercer.

Semaine 3 · Flask, Django, Express, FastAPI : quinze minutes de comparaison

	Flask	Django	Express	FastAPI
Langage	Python	Python	JavaScript	Python
Validation	à la main	DRF	bibliothèque	Pydantic
Doc auto	extension	extension	extension	intégrée
Taille du cadre	minimal	complet, lourd	minimal	léger
Bon pour	petit service	site + admin + BD	équipe JS	API de modèle

Flask : le plus simple, dix lignes pour une route ; validation, documentation et typage restent à votre charge. Très répandu, beaucoup d'exemples de ML en ligne l'utilisent.

Django : un cadre complet (ORM, admin, authentification) pensé pour un site entier ; Django REST Framework ajoute les API. Surdimensionné pour exposer un modèle.

Express : l'équivalent de Flask en Node.js ; pertinent si l'équipe est JavaScript, mais le modèle est en Python, d'où un pont à construire.

FastAPI : validation, documentation et tests intégrés, même langage que le modèle. C'est le choix du cours ; ce que vous apprenez (routes, schémas, codes, tests) se transpose aux trois autres.

Spring Boot (Java) : extension facultative du projet pour ceux qui viennent de cet écosystème.

Semaine 4 · Un test vert ne dit pas que le code est bon

intégration, modèle chargé 5

modèle, jeu d'or 6

données du TP2 8

contrat HTTP, sur doublure 15

logique pure, ni HTTP ni modèle 19

propriétés (hypothesis), tous les étages 7

`python -m pytest -q` → 60 passed in 4.11s
du bas vers le haut : de plus en
plus fidèle, de plus en plus lent

Le TP3 rendait 9 tests verts. Trois défauts y survivaient, tous trouvés cette semaine :

Entrée	TP3 répond	TP4 répond
trois espaces	neg, 0,506	422
"..."	neg, 0,506	indéterminé
"abc"	neg, 0,506	indéterminé

Le troisième ne se voyait pas à l'exécution : le label renvoyé était un texte quelconque, alors que le contrat n'admet que trois valeurs. C'est mypy qui l'a dit.

Un test vert affirme une seule chose : ce test-là passe. La question utile n'est pas « combien de tests », mais « quelle partie du système chaque étage protège, et laquelle reste nue ».

Kit : code/semaine4/, 60 tests, six fichiers ; tous les chiffres du lot sortent de mesures.json.

Semaine 4 · Anatomie d'un test : un nom, trois temps, une affirmation

Avant : un test, six affirmations, un nom qui n'apprend rien.

```
def test_tout():
    assert normaliser(" a b ") == "a b" and normaliser("x") == "
        x"
    assert texte_exploitable("bon") == True
    v = predire(PipeFactice(), ["magnifique", "ennuyeux", "table"])
    assert v[0].label == "pos" and v[1].label == "neg"
    assert len(v) == 3
```

Après : le nom dit le comportement, l'échec dit lequel a cassé.

```
@pytest.mark.parametrize(("brut", "attendu"), [
    (" Un film  magnifique ", "Un film magnifique"),
    (" ", ""),
])
def test_normaliser(brut, attendu):
    assert modele.normaliser(brut) == attendu

def test_predire_s_abstient_sous_le_seuil():
    (verdict,) = modele.predire(PipeFactice(), ["une phrase sans
        mot cle"])
    assert verdict.label == modele.INDETERMINE
    assert not verdict.tranche
```

Les trois temps : préparer l'entrée, appeler une seule fois, affirmer. Un test qui appelle deux fois mesure deux choses, et son échec ne désigne plus rien.

Le nom est le message d'erreur. `test_tout` en rouge n'apprend rien ;
`test_predire_s_abstient_sous_le_seuil` en rouge dit déjà quoi corriger.

Une affirmation par test. Avec `and`, la deuxième moitié n'est pas évaluée quand la première échoue : la moitié du test reste invisible.

Mesure : ruff relève **5 remarques** sur le fichier « avant » (PT018 assertions composées, E712 comparaison à `True`) et **aucune** sur la version corrigée. Les deux versions passent au vert : la différence n'est pas le résultat, c'est ce qu'on apprend quand il devient rouge.

`parametrize` remplace des tests jumeaux par un tableau de cas : dans le kit, 3 blocs produisent 12 des 60 tests.

Semaine 4 · Les outils de pytest : portée, paramétrage, marqueur

la portée : combien de fois la préparation est payée

pipe : session

client : module

client_double : test



déroulement de la suite, 60 tests

```
$ python -m pytest -q                # 60 passed in 4.11s
$ python -m pytest -m "not lent" -q   # 49 tests, sans le vrai modele
$ python -m pytest -m lent            # 11 passed in 3.95s
```

La portée est un budget. Charger le modèle une fois par test rendrait la suite inutilisable ; le kit a 2 fixtures de session, 1 de module, 1 par test. Règle : la portée la plus large que l'isolation autorise.

parametrize sépare les cas de la mécanique : ajouter un cas, c'est ajouter une ligne, et l'échec nomme le cas fautif (`test_normaliser[ -]`).

`tmp_path` donne un dossier jetable : `test_colonne_absente_signalee` y écrit un CSV cassé, le vérifie, et ne laisse rien derrière lui.

`monkeypatch` remplace une valeur le temps d'un test et la restaure ensuite. Sans lui, un test qui vide état casserait tous les suivants.

Les marqueurs séparent la boucle courte du contrôle

complet : `-m "not lent"` pendant qu'on code, tout en CI.

`-strict-markers` refuse un marqueur mal orthographié, sans quoi le filtre ignore silencieusement les tests.

Semaine 4 · La doublure : exercer l'API sans le modèle



ni fichier, ni entraînement, ni surprise : 0,31 s



modèle du TP2 reconstruit si absent : 3,95 s

```
class PipeFactice:
    classes_ = np.array(["neg", "pos"])
    def predict_proba(self, textes):
        lignes = []
        for bas in (t.lower() for t in textes):
            if any(m in bas for m in POSITIFS): lignes.append([0.10, 0.90])
            elif any(m in bas for m in NEGATIFS): lignes.append([0.90, 0.10])
            else:
                lignes.append([0.48, 0.52])
        return np.array(lignes)
```

Une doublure imite l'interface, pas l'intelligence. `classes_` et `predict_proba` suffisent : l'API ne demande rien d'autre au modèle.

Ce qu'on y gagne vraiment n'est pas la vitesse, c'est l'indépendance. Les 15 tests de contrat tournent sur un dépôt fraîchement cloné, sans `sentiment.joblib`, sans `scikit-learn` entraîné, et donnent toujours le même résultat.

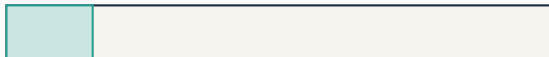
Une doublure doit être ennuyeuse. 0,90 pour un mot positif, 0,52 sinon : aucun test n'a besoin de deviner un nombre, et 0,52 place volontairement le cas sous le seuil d'abstention.

La limite : une doublure ne dit rien de la qualité du modèle.

D'où les 11 tests lents, et la règle : tout ce qui touche au modèle se teste avec le modèle.

Semaine 4 · Tester le jeu avant d'accuser le modèle

avis_demo.csv du TP2 : 290 lignes, 145 pos / 145 neg



46 lignes en doublon (22 avis répétés)



16 lignes : 8 avis portent *les deux* étiquettes, plafond 0,972



train/test : 5 des 58 lignes de test étaient déjà dans le train

exactitude sur ces 5 : 0,800 ; sur les 53 autres : 0,962

Quatre contrôles, quatre tests (`app/donnees.py`) : la forme du fichier, les doublons, les étiquettes contradictoires, et la fuite entre entraînement et test.

Le F1 de 0,948 du TP2 avait un plafond. Huit avis figurent avec pos et neg : aucun modèle ne peut avoir raison sur les deux. Chercher à dépasser 0,972 sur ce jeu, c'est chercher à apprendre du bruit.

Contre-intuitif : ici les lignes qui fuient sont *moins* bien classées (0,800) que les autres (0,962), parce que ce sont précisément les avis contradictoires. Une fuite ne gonfle pas toujours le score ; elle rend surtout le score illisible.

Le correctif tient en une ligne : dédupliquer avant de séparer. 290 lignes deviennent 266, et la fuite tombe à 0. Le test le vérifie à chaque exécution.

`train_test_split` sépare des lignes, pas des textes : il ignore qu'un même avis peut apparaître deux fois.

Semaine 4 · Le jeu d'or : il peut se taire, il ne doit pas se tromper

16 avis clairs (8 pos, 8 neg), confiance mesurée

0.575	0.913	0.905	0.593	0.611	0.927	0.509	0.580
0.799	0.885	0.893	0.756	0.735	0.836	0.607	0.578

4 avis volontairement vagues, seuil retenu : 0,60

0.506	0.553	0.506	0.505
-------	-------	-------	-------

sous lui, le label vaut
indetermine



11 justes



5 abstentions coûteuses



4 silences justes

0 réponse fausse

Un jeu d'or est écrit à la main, avec l'étiquette qu'un correcteur humain donnerait, y compris « je ne sais pas ». Vingt avis suffisent : ils servent à surveiller un comportement, pas à mesurer une performance.

Jamais extrait du jeu d'entraînement. Un jeu d'or tiré des données d'apprentissage ne mesure plus rien : il redonne ce que le modèle a déjà vu.

Le test central exige *zéro réponse fausse* : `test_aucune_reponse_fausse`. Il passe. Le contrat produit est tenu : le modèle se tait plutôt que de se tromper.

Le prix est visible : sur 16 avis clairs, il n'en tranche que 11. Cinq avis parfaitement lisibles par un humain (« Comédie drôle et touchante », 0,509) restent sans réponse. Ce chiffre est la vraie mesure du modèle, et l'objet du travail au TP4.

Un test de jeu d'or qui échoue ne dit pas « le code est cassé » mais « le modèle a changé de comportement » : à lire, pas à faire passer de force.

Semaine 4 · Le seuil n'est pas un réglage, c'est une décision

58 avis de test du TP2, jamais vus à l'entraînement

seuil	0,50	0,54	0,56	0,58	0,60	0,62	0,65	0,70	0,75
avis traités	58/58	57/58	53/58	52/58	52/58	51/58	49/58	41/58	26/58
exactitude	0,948	0,947	0,981	0,981	0,981	1,000	1,000	1,000	1,000
abstentions	0	1	5	6	6	7	9	17	32

colonne encadrée : le seuil retenu dans le kit (`SEUIL_ABSTENTION = 0.60`)

Sans seuil (0,50), le modèle répond toujours : 58 avis sur 58, et se trompe 3 fois. Avec 0,62, il ne se trompe plus jamais, mais laisse 7 avis sans réponse.

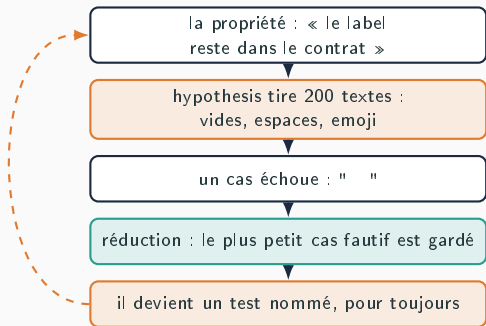
Aucune des deux colonnes n'est « la bonne ». Le choix dépend de l'usage : un tableau de bord interne préfère répondre à tout, une décision envoyée au client préfère se taire. Le cours retient 0,60 : 52 avis traités, exactitude 0,981, 6 silences.

Ce qui doit être testé n'est donc pas la valeur du seuil, mais le fait qu'elle soit *lue* quelque part et exposée : la route `/model` renvoie `seuil_abstention`, et un test l'exige.

Le saut de 0,54 à 0,56 vaut d'être regardé : 4 avis écartés, exactitude de 0,947 à 0,981. Ces 4 avis sont ceux que le modèle classait presque au hasard.

Courbe recalculée par python `mesures.py` : elle change si le modèle change, et c'est le but.

Semaine 4 · Les propriétés : ne plus choisir les exemples



```
@given(st.text(min_size=0, max_size=200))
def test_le_label_appartient_toujours_au_contrat(texte):
    (verdict,) = modele.predire(PIPE, [texte])
    assert verdict.label in {"pos", "neg", modele.INDETERMINE}
```

Un exemple teste un cas; une propriété teste une règle. On n'écrit plus « pour ce texte, ce label », mais « pour tout texte, le label reste dans le contrat » : la bibliothèque cherche elle-même le contre-exemple.

C'est cet étage qui a trouvé le défaut du TP3. La propriété « un texte d'espaces n'est jamais exploitable » a échoué sur " " : trois espaces passaient `min_length=3` et l'API répondait `neg` avec 0,506 de confiance.

La réduction est le vrai service rendu : hypothesis rétrécit le cas trouvé jusqu'au plus petit qui échoue encore. On lit " ", pas 180 caractères aléatoires.

Les sept propriétés du kit : normalisation idempotente, aucun espace aux bords, texte blanc jamais exploitable, confiance dans [0 ; 1], label dans le contrat, un verdict par texte, espaces ajoutés sans effet.

Ce que ça ne remplace pas : une propriété dit ce qui reste vrai, jamais si le modèle a raison.

Semaine 4 · Couverture, style, types : ce que les outils ne voient pas



la CI enchaîne les quatre et refuse la fusion sous 90 % (fail_under).

couverture avant les tests d'intégration



app/main.py : 85 %, lignes 25-30 = le lifespan



après 5 tests d'intégration : main.py à 100 %

93 % de couverture, démarrage non testé. Les six lignes manquantes de main.py étaient le lifespan : exactement le chargement du modèle, c'est-à-dire l'endroit où la production tombe. Un bon taux n'indique pas où sont les trous.

Ce que la couverture mesure : les lignes exécutées. **Ce qu'elle ne mesure pas :** ce qui a été vérifié. Un test sans aucune assertion couvre autant de lignes qu'un bon test.

Les 4 % restants sont nommés : app/donnees.py lignes 29, 57 et 69-71, deux branches défensives et la fonction rapport. Les couvrir fait partie du TP4.

Sur le fichier « avant », ruff relève **10 remarques** en 0,2 s (import inutile, == None, argument par défaut mutable) et mypy **2 erreurs**. Aucun des deux ne voit les deux vrais défauts : modèle rechargé à chaque appel, et absence d'abstention.

Le partage du travail : les outils tiennent la forme, les tests tiennent le comportement. Aucun ne remplace l'autre.

Semaine 4 · TP4 : durcir les tests de l'API du TP3 (2 h)

A. Faire échouer votre API du TP3 : trois espaces, texte hors sujet (20 min)



B. Sortir la logique des routes dans `app/modele.py`, avec seuil (25 min)



C. Les étages manquants : unitaire, doublure, données, jeu d'or (40 min)



D. `-cov`, un test du `lifespan`, `ruff`, `mypy`, CI (35 min)



E. `docs/tp4.md` : taux, lignes non couvertes, pourquoi

Vous partez de votre dépôt du TP3 (9 tests verts) et du kit de la semaine, qui montre les cinq étages sur 60 tests.

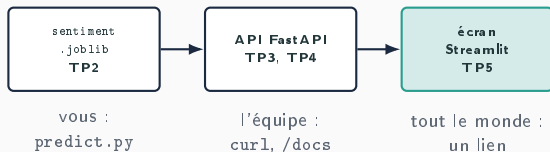
Vous rendez : le dépôt avec vos cinq fichiers de tests, votre propre jeu d'or de 20 avis dont 4 volontairement vagues, la CI qui lance `ruff`, `mypy` et `pytest -cov` avec un plancher, et `docs/tp4.md`.

Grille (sur 10) : les deux tests de la partie A passent après correction (2) ; cinq étages présents, doublure comprise (2) ; jeu d'or personnel avec zéro réponse fausse exigée (2) ; couverture mesurée et lignes non couvertes expliquées (2) ; `ruff` et `mypy` muets, CI verte (1) ; `docs/tp4.md` (1).

Pièges : couverture élevée sans assertion utile ; jeu d'or extrait du jeu d'entraînement ; `TestClient(app)` sans `with` en intégration ; seuil choisi sans courbe.

Kit : `code/semaine4/` et `TP4-colab.ipynb`.

Semaine 5 · Une interface : le moment où quelqu'un d'autre s'en sert



les 20 avis du jeu d'or, vus par l'écran



4 positifs, 7 négatifs, et 9 fois « je ne sais pas » : 45 % de l'écran

Jusqu'ici, le modèle n'était utilisable que par vous. Le TP3 l'a rendu callable par un programme ; le TP5 le rend utilisable par quelqu'un qui n'ouvrira jamais un terminal.

Une interface n'est pas une décoration. C'est le premier endroit où le modèle rencontre un jugement extérieur, et où ses défauts deviennent visibles : sur le jeu d'or du TP4, l'écran doit afficher neuf fois qu'il ne tranche pas.

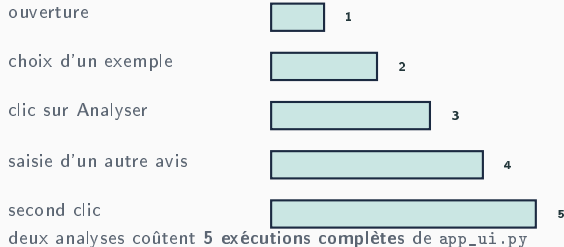
La question du lot n'est donc pas « comment faire joli », mais : que montre-t-on d'un modèle qui a raison neuf fois sur onze quand il répond, et qui refuse de répondre une fois sur deux ?

Streamlit parce que c'est du Python, que l'écran tient en 72 lignes, et qu'il se teste sans navigateur. Les mêmes idées valent pour Gradio, Dash, ou un client React devant l'API.

Kit : code/semaine5/, 49 tests dont 17 tests d'interface.

Semaine 5 · Streamlit rejoue tout le fichier, à chaque geste

compteur mesuré dans une séance de deux analyses



```
# app_ui.py : tout ce fichier est rejoue du debut a chaque
interaction
etat.initialiser(st.session_state)      # +1 au compteur a chaque
passage
texte = st.text_area("Votre avis", key="saisie")
if st.button("Analyser", key="analyser"):
    (verdict,) = obtenir_client(source).predire([texte])
```

Il n'y a pas de boucle d'événements. Cliquer ne déclenche pas une fonction : cela relance le script entier, du premier import à la dernière ligne. L'écran est reconstruit à neuf.

Trois conséquences immédiates : une variable Python ordinaire est perdue entre deux gestes ; tout calcul non protégé est refait ; et l'ordre du fichier est l'ordre de la page.

Ce modèle est un choix, pas un défaut. Il supprime la classe de bugs la plus pénible des interfaces : l'écran ne peut pas être désynchronisé de l'état, puisqu'il est recalculé à partir de lui.

Le prix est qu'il faut désigner explicitement ce qui doit survivre (`st.session_state`, slide 38) et ce qui ne doit pas être recalculé (`@st.cache_resource`, slide 39).

Le compteur est visible dans la barre latérale du kit : à observer pendant le TP.

Semaine 5 · Ce qui survit : session_state, et un piège classique

perdu à chaque geste

variables locales, résultats de calcul, position dans la page

conservé

st.session_state : historique, filtres, compteur, et la valeur de chaque widget à key

```
# AVANT : choisir un exemple ne remplissait rien
choix = st.selectbox("Exemples", list(EXEMPLES), key="exemple")
texte = st.text_area("Votre avis", value=EXEMPLES[choix], key="saisie")

# APRES : on écrit dans l'état au moment du changement
def appliquer_exemple():
    st.session_state["saisie"] = EXEMPLES[st.session_state["exemple"]]

st.selectbox("Exemples", list(EXEMPLES), key="exemple",
             on_change=appliquer_exemple)
texte = st.text_area("Votre avis", key="saisie")
```

Un widget qui a une key range sa valeur dans l'état.

Dès le deuxième affichage, c'est l'état qui gagne : l'argument value est ignoré. Le code « avant » est donc silencieusement mort.

Mesuré : après avoir choisi « Avis clairement positif », la zone de saisie contient une chaîne vide dans la version avant, et l'avis attendu après correction. mesures.py rejoue les deux versions.

Ce défaut n'apparaît pas au premier essai : à l'ouverture, l'état est vide, donc value s'applique et tout semble marcher. Il ne se voit qu'à la deuxième interaction, c'est-à-dire en démonstration devant un public.

La règle : une valeur affichée vient de l'état, jamais d'un argument recalculé. Pour la changer, on écrit dans l'état depuis un on_change.

ui/etat.py n'importe pas Streamlit : il prend un dictionnaire, donc ses six tests tournent sans interface.

Semaine 5 · Le cache : ce qu'on refuse de refaire à chaque geste

`@st.cache_resource` : un *objet* partagé, modèle chargé, connexion, client.
Rendu tel quel, jamais recopié.

`@st.cache_data` : une *valeur* recopiée à chaque appelant, tableau lu, requête, agrégat.

rien : tout ce qui dépend de la saisie en cours, et qui doit être recalculé.

```
@st.cache_resource      # charge une fois, pas a chaque re-  
                        execution  
def construire_client(source):  
    return ClientAPI(API_URL) if source == "API" else  
        ClientLocal(MODELE)
```

Mesure sur le kit : charger le modèle du TP2 coûte 1,4 ms, et une séance de deux analyses déclenche 5 ré-exécutions. Sans cache, on paierait donc 5 chargements au lieu d'un, soit 7 ms perdus.

Sept millisecondes, ce n'est rien : le modèle du cours est minuscule. C'est justement pourquoi il faut retenir le mécanisme et non le chiffre. Le même écran devant un modèle de langue chargerait le modèle à chaque clic, et l'interface deviendrait inutilisable.

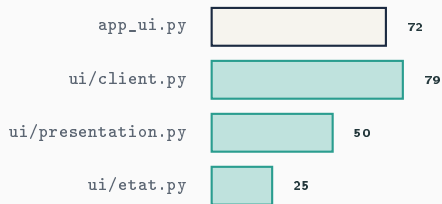
Le bon critère n'est pas « est-ce lent » mais « est-ce que cet objet doit être unique ». Un modèle, une connexion, un client HTTP : `cache_resource`. Un tableau de résultats : `cache_data`.

Le piège : `cache_data` recopie ce qu'il rend. Y mettre un modèle le sérialiserait à chaque appel, plus lentement que de ne rien cacher.

La clef du cache est faite des arguments : les deux sources sont deux entrées distinctes, donc basculer ne recharge rien.

Semaine 5 · L'écran affiche, il ne décide pas

lignes de code, commentaires exclus



l'écran : 72 lignes de widgets ; la logique qu'il appelle : 155 lignes, testables sans Streamlit

```
class Client(Protocol):          # le contrat que l'ecran exige
    def predire(self, textes: list[str]) -> list[Verdict]: ...
    def sante(self) -> dict[str, Any]: ...

class ClientLocal: ...          # charge sentiment.joblib
class ClientAPI: ...            # appelle l'API du TP4
class ClientFactice: ...        # la doublure des tests
```

Trois implémentations, un seul contrat de deux méthodes. L'écran ne sait pas d'où vient la prédiction, donc changer de source est un choix dans la barre latérale, pas une réécriture.

Le seuil, les libellés, les couleurs et les messages d'erreur vivent dans ui/, pas dans l'écran. C'est la seule raison pour laquelle 26 des 49 tests tournent sans Streamlit du tout.

Le test décisif : si vous ne pouvez pas répondre « que se passe-t-il pour cette entrée » sans lancer l'interface, c'est qu'une décision est au mauvais endroit.

Le Protocol n'est pas une formalité : déclaré runtime_checkable, il permet au test test_la_doublure_respecte_le_contrat d'échouer le jour où la doublure prend du retard sur les vrais clients.

Même découpage qu'en semaine 4 : l'API traduisait en HTTP, l'écran traduit en pixels.

Semaine 5 · Afficher « je ne sais pas » sans avoir l'air en panne

Avis positif

90 %

Le vocabulaire penche clairement du côté positif.

Modèle : local:sentiment.joblib

Le modèle ne tranche pas

52 %

La confiance est sous le seuil : ce verdict serait un pile ou face. Seuil : 60 %, confiance obtenue : 52 %.

gris, pas rouge : ce n'est pas une erreur

Trois éléments, toujours dans le même ordre : le verdict en mots, la confiance en pourcentage, et une phrase qui dit pourquoi. Un chiffre seul ne se lit pas ; une phrase seule ne se compare pas.

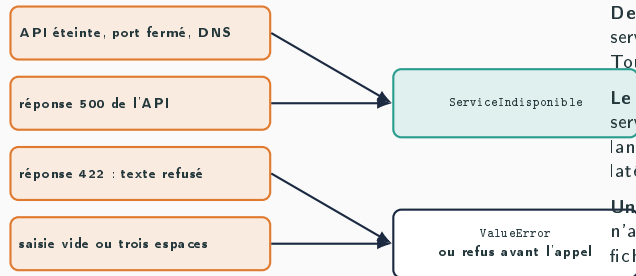
L'abstention a le même format que les autres réponses. Si elle apparaissait en rouge ou dans une boîte d'erreur, l'utilisateur conclurait que l'application est cassée, et le travail de la semaine 4 serait perdu à l'affichage.

Quand le modèle se tait, on donne les deux nombres : le seuil exigé et la confiance obtenue. C'est ce qui permet à l'utilisateur de juger si 52 % contre 60 % mérite un second regard humain.

Ce n'est pas une question cosmétique : sur le jeu d'or, 9 avis sur 20 tombent dans ce cas. Le silence n'est pas un cas marginal, c'est presque la moitié de ce que l'écran montre.

La version du modèle est affichée sous chaque verdict : sans elle, personne ne peut dire de quel modèle vient une capture d'écran reçue trois semaines plus tard.

Semaine 5 · Une panne se traduit, elle ne se propage pas



une phrase, jamais une trace

```
except Exception as erreur:                # port fermé, DNS,
    delai depasse
    raise ServiceIndisponible(f"API injoignable sur {self.
        base_url}") from erreur
```

Deux familles seulement arrivent jusqu'à l'écran : « le service ne répond pas » et « votre texte ne convient pas ». Tout le reste est traduit dans l'une des deux par le client.

Le message dit quoi faire, pas ce qui s'est passé : « Le service de prédiction ne répond pas. Vérifiez que l'API est lancée ou basculez sur le modèle local dans la barre latérale. »

Une trace Python affichée à l'écran est une faute : elle n'aide pas l'utilisateur, et elle divulgue des chemins de fichiers. Le détail reste dans la console du serveur.

Le cas le plus fréquent est le moins soigné d'habitude : la saisie vide. Ici elle est refusée avant tout appel, et un test vérifie que le client n'a bien reçu aucune requête.

Trois des 17 tests d'interface portent sur ces chemins d'erreur : panne de service, texte trop court, et saisie faite uniquement d'espaces ou de tabulations.

Semaine 5 · Tester l'écran : AppTest rejoue le script

```
def lancer(client):
    app = AppTest.from_file("app_ui.py", default_timeout=30)
    app.session_state["client"] = client          # la couture d'
    injection
    app.run()
    return app

def test_une_abstention_est_affichee_comme_un_resultat():
    app = analyser(lancer(ClientFactice()), VAGUE)
    assert app.metric[0].value == "Le modele ne tranche pas"
    assert not app.error
    assert any("Seuil" in bloc.value for bloc in app.markdown)

def test_un_texte_trop_court_ne_declenche_aucun_appel():
    client = ClientFactice()
    app = analyser(lancer(client), " ")
    assert client.appels == []
    assert app.warning[0].value.startswith("Ecrivez au moins")
```

```
$ python -m pytest tests/test_apptest.py
17 passed in 1.28s
```

AppTest exécute le script en mémoire et expose ce qu'il a produit : `app.metric`, `app.error`, `app.warning`, `app.markdown`, `app.session_state`. Ni navigateur, ni serveur, ni capture d'écran.

Interagir, c'est trois appels : `set_value` sur un widget, `click` sur un bouton, puis `run()` qui rejoue le script exactement comme le ferait un geste réel.

On affirme ce que l'utilisateur voit, pas ce que le code fait : le libellé du verdict, l'absence de boîte rouge, la présence du seuil dans l'explication.

La doublure sert deux fois : elle rend les tests déterministes, et elle enregistre ses appels, ce qui permet d'affirmer qu'une saisie vide ne déclenche *aucune* requête.

17 tests d'interface en 1,28 s : c'est assez rapide pour tourner à chaque sauvegarde, donc assez rapide pour être vraiment lancé.

49 tests au total : 17 d'interface, 12 de clients, 14 de mise en forme, 6 d'état.

Semaine 5 · Ce que les tests d'interface ne verront jamais

vérifié automatiquement

les libellés affichés, les codes d'erreur traduits, l'état conservé, l'ordre de l'historique, le nombre d'appels au modèle, le seuil montré à l'utilisateur

49 tests, couverture de ui/ : 95 %, ruff et mypy muets

à regarder soi-même

la mise en page réelle, la lisibilité des couleurs, le comportement sur téléphone, le temps d'attente ressenti, et la question qu'aucun test ne pose : est-ce compréhensible ?

une démonstration de cinq minutes à quelqu'un d'extérieur

AppTest ne rend rien : ni pixel, ni CSS, ni navigateur. Deux éléments superposés ou un texte blanc sur fond blanc passent au vert.

Il ne mesure pas le temps ressenti : les 1,28 s de la suite ne disent rien de l'attente devant un modèle lourd, la doublure répondant instantanément.

Ce n'est pas une faiblesse à corriger mais la contrepartie de tests rapides et stables : un test par capture d'écran casse à chaque changement de police, et finit désactivé. Les tests tiennent le comportement, une revue humaine tient l'apparence.

La couverture porte sur ui/, pas sur l'écran : coverage et AppTest instrumentent tous deux app_ui.py et se gênent.

C'est une limite des outils, pas un modèle à suivre : on n'exclut jamais un fichier parce qu'il est mal couvert, et cette limitation est à revérifier à chaque montée de version.

Semaine 5 · TP5 : une interface pour le modèle (2 h)

A. Lancer, compter les ré-exécutions, retirer le cache, éteindre l'API (20 min)



B. Analyser un lot, sans mettre une seule décision dans l'écran (25 min)



C. Six tests AppTest, dont l'abstention et la panne (40 min)



D. Filtre d'historique, défaut value/key démontré, CI (35 min)



E. docs/tp5.md : ce qui a été mesuré, et ce qui a surpris

Vous partez de votre API du TP4 et du kit : un écran de 72 lignes, 155 lignes de logique testable, 49 tests verts.

Vous rendez : l'analyse par lot qui n'appelle le client qu'une fois, vos six tests AppTest, le filtre d'historique qui survit à une nouvelle analyse, la CI verte, et docs/tp5.md.

Grille (sur 10) : le lot fonctionne et n'appelle le client qu'une fois (2) ; six tests AppTest pertinents, abstention et panne comprises (3) ; filtre testé, état conservé (2) ; ruff, mypy, couverture au-dessus du plancher (1) ; docs/tp5.md avec le défaut value/key expliqué (2).

Pièges : un seuil ou un libellé glissé dans app_ui.py ; le modèle chargé sans @st.cache_resource ; une abstention affichée en rouge ; et vouloir tester en lançant un navigateur.

Colab : pas d'écran affiché, mais AppTest rejoue tout, tests et mesures compris.

Kit : code/semaine5/ et TP5-colab.ipynb.

Semaine 6 · « Ça marche chez moi » : l'import qui dépend du dossier

`python -c "import sentiment"`, lancé depuis trois endroits

à la racine du projet, sans installation

`ModuleNotFoundError`

avec `PYTHONPATH=src`

marche, ici seulement

installé, depuis n'importe quel dossier

marche partout

le même code, trois résultats : ce n'est pas le code qui décide, c'est le chemin d'import

```
$ cd /un/autre/dossier && python -c "import sentiment"
ModuleNotFoundError: No module named 'sentiment'
$ pip install sentiment-0.3.0-py3-none-any.whl
$ cd /encore/ailleurs && sentiment predire "Un film magnifique."
pos          0.759  Un film magnifique.
```

Jusqu'ici, tout tenait par le répertoire courant. Les TP2 à TP5 marchaient parce qu'on les lançait depuis le bon dossier : Python ajoute ce dossier au chemin d'import, et l'illusion tient tant que personne ne déplace rien.

Empaqueter, c'est rendre le code indépendant de l'endroit d'où on l'appelle. Après installation, `import sentiment` fonctionne depuis n'importe quel dossier, dans n'importe quel script, y compris chez quelqu'un d'autre.

Ce n'est pas qu'une commodité : tant que le code n'est pas installable, le déployer (semaine 8) ou le faire tourner en CI ailleurs que dans le dépôt relève de la chance.

Trois livrables cette semaine : un paquet installable, une commande utilisable sans écrire de Python, et une documentation générée depuis le code.

Kit : `code/semaine6/`, 40 tests, dont 9 qui construisent et installent vraiment le paquet.

Semaine 6 · src/ : rendre impossible l'import accidentel

```
code/semaine6/
|- pyproject.toml      <- identite et dependances
|- README.md          <- publie avec le paquet
|- src/
|   '- sentiment/
|       |- __init__.py  <- la surface publique
|       |- modele.py
|       |- cli.py
|       '- py.typed     <- les types sont livres
|- tests/              <- absents de la roue
|- models/             <- jamais distribues
'- mesures.py
```

à la racine, Python ne voit pas `sentiment` : il n'y a rien à cet endroit. Il faut l'installer, comme le fera l'utilisateur.

Le piège que cette disposition supprime. Si le paquet est à la racine, lancer les tests depuis la racine l'importe *sans installation* : on teste alors l'arborescence de travail, pas ce qui sera livré. Un fichier oublié dans la distribution passe inaperçu jusqu'à la première installation réelle.

Avec `src/`, cet import de complaisance est impossible. Les tests n'ont que deux façons de trouver le paquet : l'installer, ou dire explicitement où il est. Le kit choisit la seconde pour les tests rapides (`pythonpath = ["src"]` dans `pyproject.toml`) et la première pour les tests de distribution.

Ce que la roue contient est décidé ici : `tests/`, `models/` et `mesures.py` sont en dehors de `src/`, donc dehors par construction, sans liste d'exclusion à maintenir.

`py.typed` est un fichier vide qui change tout pour l'utilisateur : sans lui, `mypy` ignore les annotations du paquet installé, et le typage s'arrête à la frontière.

Semaine 6 · pyproject.toml : un seul fichier décrit le projet

```
[build-system]                # avec quoi on construit
requires = ["hatchling"]
build-backend = "hatchling.build"

[project]                     # l'identité du paquet
name = "sentiment"
version = "0.3.0"
requires-python = ">=3.11"
readme = "README.md"
dependencies = [              # ce qu'il faut POUR S'EN SERVIR
    "joblib>=1.3",
    "scikit-learn>=1.3",
]

[project.optional-dependencies] # ce qu'il faut POUR LE
    DEVELOPPER
dev = ["pytest>=8", "pytest-cov", "ruff", "mypy", "build", "pdoc"]

[project.scripts]             # crée la commande à l'
    installation
sentiment = "sentiment.cli:main"

[tool.pytest.ini_options] [tool.ruff.lint] [tool.mypy] # et les
    outils
```

Quatre blocs, quatre questions : avec quoi construire, qui est ce paquet, de quoi il a besoin, et ce qu'il installe en plus du code.

La distinction la plus utile est celle entre `dependencies` et `optional-dependencies`. Qui installe le paquet reçoit `joblib` et `scikit-learn`, pas `pytest` ni `ruff`. Un test du kit le vérifie : dans l'environnement neuf, `import pytest` échoue.

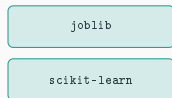
Les bornes de version sont des promesses : `>=1.3` dit « je sais fonctionner à partir de là ». Figer une version exacte dans une bibliothèque rend son installation impossible à côté d'un autre paquet.

Le même fichier configure les outils (`pytest`, `ruff`, `mypy`, `coverage`) : plus de `setup.py` ni de fichier par outil.

Le moteur de construction est `hatchling`, déclaré dans `[build-system]` et utilisé tel quel par le kit. `setuptools` est l'alternative historique : même rôle, même section, et le plus répandu encore aujourd'hui.

Semaine 6 · Deux dépendances déclarées, huit paquets installés

déclaré dans `pyproject.toml`



2 lignes écrites, **8 paquets** dans l'environnement : 6 arrivent sans avoir été demandés

aucun outil de développement : `import pytest` échoue dans cet environnement, et un test l'exige

réellement installé



Une dépendance n'est jamais seule. `scikit-learn` amène `numpy`, `scipy`, `threadpoolctl` et le reste. Chacun est du code que vous ne relierez pas, qui peut changer, et qui devra être téléchargé à chaque déploiement.

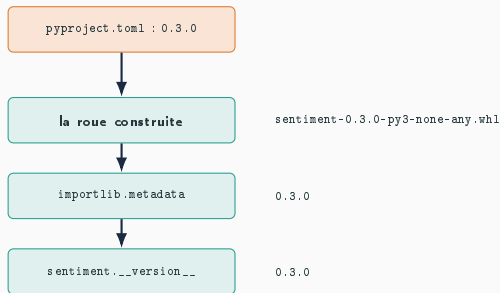
D'où la règle : ne déclarez que ce que vous importez vraiment. Recopier le `pip freeze` de votre machine fige des paquets que vous n'utilisez pas et rend l'installation fragile.

Le test qui garde la frontière : après installation de la roue seule, l'environnement contient 8 paquets et *aucun* outil de développement. Si quelqu'un glisse `pytest` dans `dependencies`, ce test devient rouge.

À l'inverse, une dépendance oubliée ne se voit jamais chez vous, puisqu'elle est installée. Elle ne se voit que chez l'utilisateur, à l'import. C'est exactement ce que les tests d'installation en environnement neuf détectent.

Mesuré par `mesures.py` : `pip list` avant et après l'installation de la roue.

Semaine 6 · La version : écrite une fois, relue partout



hors installation, la même ligne rend `0.0.0+source` : la version n'existe qu'une fois le paquet installé, et c'est honnête

```
from importlib.metadata import PackageNotFoundError, version
try:
    __version__ = version("sentiment")
except PackageNotFoundError:  # execute depuis les sources
    __version__ = "0.0.0+source"
```

Le numéro n'est écrit qu'à un seul endroit, dans `pyproject.toml`. Le code le *relit* depuis les métadonnées installées. Rien n'est recopié, donc rien ne peut diverger.

L'erreur classique est un `__version__ = "0.3.0"` écrit en dur dans le code *en plus* de `pyproject.toml`. Le jour où l'un des deux est mis à jour seul, la roue et le paquet importé annoncent deux versions différentes, et plus personne ne sait ce qui tourne.

Le repli `0.0.0+source` n'est pas un pansement : il dit exactement la vérité, à savoir que ce code n'a pas été installé et que sa version n'a donc pas de sens. Mieux vaut cela qu'un numéro rassurant et faux.

Pourquoi la version compte ici : à partir de la semaine 7, un modèle en production sera identifié par le couple version du paquet et version du modèle. Une capture d'écran de résultat sans ces deux numéros n'est pas exploitable.

Semaine 6 · La surface publique est une promesse

public : 8 noms

```
charger, predire, normaliser, lire_version_modele, Verdict, SEUIL_ABSTENTION,
INDETERMINE, __version__
```

documentés : 8 sur 8, et c'est un test

interne : 13 noms

```
verifier_classes, texte_exploitable, CLASSES_ATTENDUES, Label,
LONGUEUR_MINIMALE, les imports du module...
```

absents de la documentation, modifiables sans prévenir

`__all__` n'est pas décoratif. C'est la liste de ce que vous vous engagez à ne pas casser. Tout ce qui n'y est pas peut être renommé, déplacé ou supprimé sans prévenir personne.

Élargir est facile, réduire casse. Ajouter un nom public n'ennuie personne ; en retirer un casse le code de ceux qui l'utilisaient. D'où un test qui compare `__all__` à la liste attendue : le réduire devient un acte délibéré, pas un accident de refactorisation.

L'utilisateur écrit `from sentiment import predire`, jamais `from sentiment.modele import predire`. Le chemin interne reste libre : `modele.py` peut être découpé demain sans casser personne.

Conséquence directe sur la documentation : `pdoc` ne documente que `__all__`. Les 13 noms internes n'apparaissent pas dans la page produite, et un test le vérifie. La surface publique pilote donc à la fois le contrat et la documentation.

Semaine 6 · Une commande installée, des codes de sortie utiles

```
[project.scripts]
sentiment = "sentiment.cli:main"
```

```
$ sentiment --json predire "Les acteurs sont excellents."
[{"texte": "...", "label": "pos", "confiance": 0.913}] -> 0
$ sentiment --json predire "Correct sans plus."
[{"texte": "...", "label": "indetermine", ...}] -> 0
$ sentiment predire " "
Au moins trois caracteres utiles par avis. (stderr) -> 1
$ sentiment --modele absent.joblib predire "Un film."
Modele absent : absent.joblib. (stderr) -> 2
```

```
def main(argv: list[str] | None = None) -> int: # rend le code
    , n'appelle pas sys.exit
```

Le point d'entrée transforme une fonction en commande. À l'installation, pip fabrique un exécutable sentiment qui appelle `sentiment.cli:main`. L'utilisateur n'écrit plus de Python, et ne connaît pas le chemin du fichier.

`main` rend un entier, elle n'appelle pas `sys.exit`. C'est ce qui permet de l'appeler directement dans un test et de vérifier le code de sortie, sans sous-processus ni capture d'exception.

Trois codes distincts, pas un seul : 0 succès, 1 entrée refusée, 2 ressource absente. Un script qui enchaîne des commandes peut alors réagir : réessayer, prévenir, ou s'arrêter.

Les résultats sur la sortie standard, les erreurs sur la sortie d'erreur. C'est ce qui rend `sentiment --json lot avis.txt` > `resultats.json` utilisable : le fichier ne contient que des données, jamais un message d'erreur.

Une abstention n'est pas une erreur : elle sort en code 0, avec le label `indetermine`. Le contrat de la semaine 4 se propage jusqu'au terminal.

Semaine 6 · Construire, et regarder ce qu'on distribue vraiment

	roue (.whl)	archive (.tar.gz)
poids	6,5 Ko	≈ 10 Ko
fichiers	8	14
modules du paquet	4	4
tests inclus	0	6

la roue s'installe telle quelle; l'archive source permet de reconstruire et de relancer les tests chez quelqu'un d'autre

```
$ python -m build
Successfully built sentiment-0.3.0.tar.gz
                        and sentiment-0.3.0-py3-none-any.whl
$ unzip -l dist/*.whl | grep sentiment/
sentiment/__init__.py      sentiment/modele.py
sentiment/cli.py           sentiment/py.typed
```

Deux distributions, deux usages. La roue est le paquet prêt à poser : pip la décompresse, rien n'est exécuté. L'archive source contient de quoi reconstruire, donc aussi les tests.

Ce qui n'est pas dans la roue est aussi important que ce qui y est. Zéro fichier de test, zéro .joblib. Un modèle entraîné n'a rien à faire dans une distribution de code : il change à un autre rythme, et il pèse souvent mille fois plus.

Quatre tests gardent ce contenu : la roue contient les quatre modules et `py.typed`, elle ne contient ni test ni modèle, l'archive source contient les tests, et le nom de la roue porte la version déclarée.

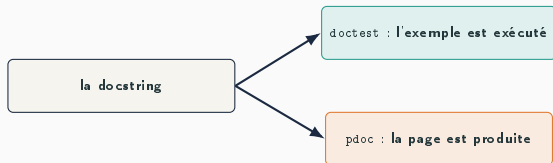
Pourquoi tester cela : un fichier oublié dans la distribution ne se voit jamais chez soi, puisque le fichier est là. Il se voit à la première installation ailleurs, c'est-à-dire trop tard.

`dist/` n'est pas versionné : une distribution se reconstruit, elle ne se commite pas.

Semaine 6 · Une documentation qui ne peut pas mentir longtemps

```
def normaliser(texte: str) -> str:
    """Rduit toute suite d'espaces a un espace et coupe les bords.

    >>> normaliser("  Un film    magnifique  ")
    'Un film magnifique'
    """
    return " ".join(texte.split())
```



```
$ python -m pdoc sentiment -o site/      # 8 noms publics, 13
      internes absents
$ python -m pytest tests/test_documentation.py  # 14 passed
```

Une documentation écrite à côté du code diverge en quelques semaines. Écrite *dans* le code et exécutée par les tests, elle ne le peut pas : un exemple faux devient un test rouge.

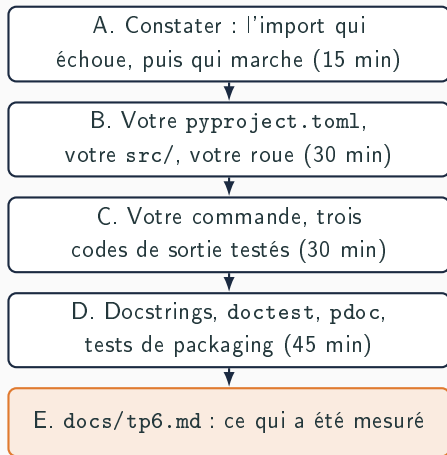
Quatre choses sont vérifiées automatiquement : chaque nom public a une docstring utile, chaque module en a une, les exemples »> donnent le résultat annoncé, et les sous-commands promises par le README existent dans le parseur.

Le README est testé lui aussi : renommer une commande sans toucher au README rend le test rouge.

La page produite suit `__all__` : 8 noms publics documentés, 13 noms internes absents. Documenter n'est donc pas une tâche en plus, c'est une conséquence d'avoir déclaré ce qui est public.

site/ n'est pas versionné : la documentation se régénère, comme la roue.

Semaine 6 · TP6 : emballer et documenter le modèle (2 h)



Kit : `code/semaine6/` et `TP6-colab.ipynb`.

Vous partez de vos scripts des TP2 à TP5, et du kit : 40 tests, dont 9 qui construisent et installent vraiment le paquet.

Vous rendez : un dépôt dont `python -m build` produit roue et archive source, une commande installée à trois codes de sortie, une documentation générée, et `docs/tp6.md`.

Grille (sur 10) : `build` produit les deux distributions (2) ; commande installée, trois codes testés (2) ; quatre tests de packaging pertinents (2) ; docstrings publiques, doctest vert, documentation générée (2) ; ruff avec les règles D, mypy, couverture (1) ; `docs/tp6.md` (1).

Pièges : recopier le numéro de version dans le code ; mettre le paquet à la racine plutôt que dans `src/` ; déclarer en dépendance tout ce qui traîne sur votre machine ; embarquer le `.joblib` dans la roue ; versionner `dist/`.

Colab installe réellement le paquet : la commande `sentiment y` est disponible comme sur une machine.

Semaine 7 · « C'était quelle configuration, déjà ? »

ce qu'on retient d'une séance d'essais

« le deuxième était meilleur »
« j'ai changé un truc dans le tfidf »
« 0,94 quelque chose, je crois »
trois essais, aucune trace exploitable

ce que la séance a vraiment produit

6 configurations enregistrées, chacune avec ses paramètres, sa graine, le nom du jeu, et quatre mesures ; écart du meilleur au pire : 0,0174 de F1
relisible dans six mois, par quelqu'un d'autre

Le problème n'est pas de produire des scores, c'est de pouvoir y revenir. Un score sans sa configuration n'est pas un résultat : c'est une anecdote, invérifiable et non reproductible.

Trois questions reviennent toujours, et aucune ne se répond de mémoire : quelle configuration a donné ce chiffre, sur quelles données, et est-ce que je saurais le refaire à l'identique.

Deux outils, deux questions distinctes. MLflow garde les expériences : paramètres, métriques, artefacts. DVC garde les données : quelle version du jeu était en place. Les confondre est l'erreur classique de la semaine.

Ce n'est pas de la bureaucratie. Au TP2, le F1 de 0,948 ne disait rien des huit avis contradictoires du jeu. Cette semaine, le score et les données qui l'ont produit voyagent ensemble.

Kit : code/semaine7/, 23 tests, MLflow 3.16 et DVC 3.67.

Semaine 7 · Ce qu'un run doit contenir pour servir plus tard

```
with mlflow.start_run(experiment_id=identifiant) as run:
    mlflow.log_params(asdict(config))      # ngram_max, min_df, C
    ...
    mlflow.log_param("graine", GRAINE)    # sans elle, rien n'
        est refaisable
    mlflow.log_param("jeu", Path(csv).name)
    mlflow.log_metrics(metrics)            # f1, exactitude,
        duree, vocabulaire
    mlflow.set_tag("etape", "semaine7")
```

paramètres : ce que j'ai choisi

graine : ce qui fige le hasard

jeu : sur quoi j'ai mesuré

métriques : ce que j'ai obtenu

étiquettes : pour retrouver

artefacts : modèle, courbes

retirez-en un seul, et le run cesse d'être reproductible

Un run est un enregistrement complet, pas un score. Les paramètres disent *quoi*, la graine dit *comment le refaire*, le nom du jeu dit *sur quoi*, les métriques disent *le résultat*.

La graine est le paramètre qu'on oublie. Sans elle, deux runs identiques sur le papier donnent deux chiffres différents, et toute comparaison devient du bruit. Le kit la fixe à 42 et un test l'exige.

Enregistrez au moins une métrique qui n'est pas un score : ici la durée d'entraînement et la taille du vocabulaire. C'est ce qui permettra, au slide suivant, de départager des configurations à score égal.

Le découpage du code suit celui des TP précédents : évaluer est une fonction pure qui entraîne et mesure, enregistrer l'appelle et écrit dans MLflow. Les six tests de déterminisme portent sur la première, sans toucher au suivi.

Semaine 7 · Six configurations, et ce que le suivi en dit

ngram	min_df	C	F1 macro	vocabulaire
3	1	1,0	0,9481	917
2	2	1,0	0,9481	281
1	1	1,0	0,9481	92
2	1	1,0	0,9481	411
2	1	10,0	0,9310	411
2	1	0,1	0,9307	411

écart du meilleur au pire : 0,0174 de F1. Les six runs tiennent dans 3,9 s et une base SQLite de 856 Ko.

Un seul réglage déplace le score : le paramètre de régularisation C. Le passer à 0,1 ou à 10 fait perdre environ 0,017 de F1. Tout le reste, c'est-à-dire ce qu'on passe d'habitude le plus de temps à régler, ne change rien du tout.

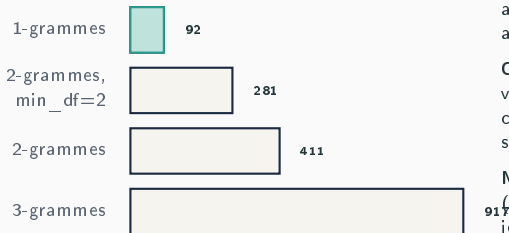
Ce constat n'est lisible que parce que les six runs sont côte à côte. Menés un par un sur trois jours, avec les résultats notés à la main, il serait passé inaperçu.

Le tableau se relit par programme, pas seulement à l'œil : `charger_runs` rend un tableau trié, meilleur désigne la première ligne, `ecart` mesure l'amplitude. Sept tests portent sur cette relecture.

Attention à la sur-interprétation : 0,0174 de F1 sur 58 avis de test, c'est environ un avis. Le suivi ne transforme pas un petit jeu en preuve.

Semaine 7 · Quatre gagnants à égalité : sur quoi trancher ?

quatre configurations, exactement le même F1 de 0,9481



à score égal, le modèle de 92 mots fait le même travail que celui de 917

Le suivi ne désigne pas un gagnant, il montre un paysage. Ici, quatre configurations sur six atteignent le même score au millième près. Prendre « la première de la liste » serait arbitraire.

C'est la métrique « inutile » qui tranche : la taille du vocabulaire n'était pas un score, elle décide pourtant du choix final. Le cours retient le plus petit modèle, qui s'inspecte à la main.

Mais ces quatre F1 viennent d'un seul découpage (random_state=42). Le slide 14b l'a mesuré sur ce même jeu : en validation croisée, le F1 va de 0,878 à 0,965, écart-type 0,029. Une égalité au quatrième chiffre ne survit pas à ce bruit.

La suite logique est de rejouer la grille en validation croisée : l'égalité tient-elle sur cinq découpages, ou n'était-ce qu'une coïncidence de celui-ci ?

Un test exige qu'il reste des ex aequo aux vocabulaires différents : si le modèle change, le slide cesse de mentir.

Semaine 7 · Sans graine fixée, comparer deux runs ne veut rien dire

trois sources de hasard dans une seule expérience

`train_test_split` : **quelles lignes partent en test**

`LogisticRegression` : **l'initialisation du solveur**

l'ordre des données, s'il dépend du système de fichiers

`GRAINE = 42` **passée aux deux premiers, et enregistrée dans le run**

```
def test_deux_evaluations_identiques_donnent_le_meme_score(jeu):  
    premiere = evaluer(Configuration(), jeu)  
    seconde = evaluer(Configuration(), jeu)  
    assert premiere["f1_macro"] == seconde["f1_macro"]
```

L'égalité est stricte, pas une approximation. Deux appels identiques doivent rendre le même nombre, chiffre pour chiffre. Dès qu'il faut écrire approx, c'est qu'une source de hasard subsiste quelque part.

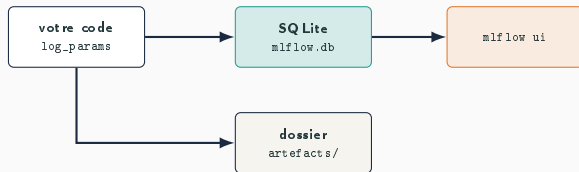
Pourquoi c'est la condition de tout le reste : l'écart mesuré au slide 58 vaut 0,0174. Si deux runs de la même configuration variaient de 0,01, la moitié du classement serait du bruit, et le gagnant changerait à chaque exécution.

La graine s'enregistre comme un paramètre, pas comme une constante cachée dans le code. Un run qui ne dit pas sa graine ne se rejoue pas, même si le code est resté le même.

Le déterminisme a des limites honnêtes : il garantit qu'on refait le même calcul, pas que le résultat est vrai. Un modèle reproductible peut être reproductiblement mauvais.

Dix tests portent sur les expériences, dont six exécutables sans lancer la grille complète.

Semaine 7 · Où vivent les runs, et comment les relire



six runs, paramètres et métriques compris : une base de 856 Ko. Les artefacts lourds (modèles, figures) vivent à côté, pas dans la base.

```
$ python -m mlflow ui --backend-store-uri sqlite:///entrepot/  
mlflow.db
```

```
mlflow.set_tracking_uri(f"sqlite:///{racine.as_posix()}/mlflow.  
db")
```

Depuis MLflow 3, le dossier `mlruns/` est refusé par défaut. L'entrepôt « fichier » est en fin de vie ; le backend attendu est une base. Le kit utilise un SQLite local, et le message d'erreur qui l'a imposé est reproductible : lancez la grille sans changer l'URI pour le voir.

Ce n'est pas qu'une contrainte : le même code pointe vers un serveur d'équipe en changeant une seule chaîne. Le passage du poste personnel au serveur partagé ne demande aucune réécriture.

Deux stockages, deux natures. La base garde ce qui se trie et se compare, quelques kilo-octets. Les artefacts gardent ce qui est lourd et opaque. Mélanger les deux fait une base de plusieurs giga-octets, lente à interroger.

L'interface web n'est pas l'outil principal : elle sert à explorer. Ce qui compte pour le TP, c'est `search_runs`, qui rend un tableau relisible par programme, donc testable.

Mesuré avec MLflow 3.16.1 ; l'API de suivi change peu, l'entrepôt par défaut change souvent.

Semaine 7 · Ce que Git sait faire, et là où il s'arrête

Git est très bon pour

du texte qui change par petites touches : code, configuration, documentation. Il stocke les différences, pas les copies.

ajouter un avis au CSV : 1 ligne de diff

Git s'arrête devant

les fichiers qui changent en profondeur ou qui pèsent : chaque version est conservée, et chaque clone la retéléchargera, pour toujours.

régénérer le jeu : 512 lignes de diff

La question n'est pas « Git ou DVC », mais « qu'est-ce qui change, et comment ». Un CSV auquel on ajoute des lignes reste du texte : Git le gère parfaitement, et mieux que DVC.

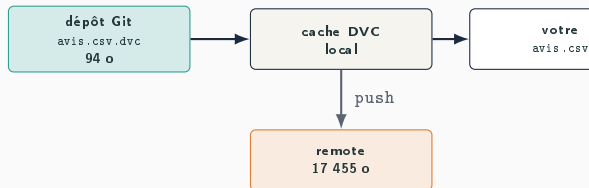
Trois situations font basculer : le fichier est binaire (une archive, un modèle, des images) ; il est réécrit entièrement à chaque mise à jour ; ou il pèse assez pour que son historique devienne un fardeau.

Le point irréversible : ce qui entre dans l'historique Git y reste. Un fichier de 500 Mo commité puis supprimé pèse toujours 500 Mo dans chaque clone. Réécrire l'historique d'un dépôt partagé est une opération douloureuse.

Le critère pratique : si vous hésitez à faire `git add` sur un fichier parce qu'il est gros, c'est le signal. À ce moment-là seulement, DVC devient le bon outil.

Les deux chiffres ci-contre sont mesurés sur le jeu du TP2 par `mesures.py` ; le slide suivant montre le même essai côté DVC.

Semaine 7 · DVC : le dépôt garde l’empreinte, pas les octets



Le fichier `.dvc` tient en quatre lignes : une empreinte md5, une taille, un chemin. C’est lui, et lui seul, qui entre dans Git. Les octets partent dans un cache local, puis vers un remote.

Le remote n’a rien d’un service payant : ici c’est un simple dossier. En équipe, ce sera un partage réseau, un S3 ou un Drive. Le mécanisme est identique, et le TP le fait en local.

`dvc add` écrit aussi un `.gitignore`. Le CSV devient donc impossible à commiter par mégarde : c’est la protection qui manque quand on se contente d’une consigne d’équipe.

Le test qui compte n’est pas `dvc push`, c’est `dvc pull` après suppression. Un dépôt où le pointeur voyage sans que les octets aient été poussés donne à votre binôme un renvoi vers rien. Six tests rejouent ce cycle complet.

Empreinte du jeu du TP2 : `6563b2f4...`, identique avant et après le cycle.

le CSV pèse 186 fois le pointeur que Git transporte

```
$ dvc add data/avis.csv      # cree avis.csv.dvc et un .
                             gitignore
$ dvc push                  # 1,25 s vers le remote
$ rm data/avis.csv          # le fichier disparaît
$ dvc pull                  # 1,92 s : 17 455 octets, empreinte
                             identique
```

Semaine 7 · DVC gagne, sauf quand il perd

	Git seul	avec DVC
ajouter un avis	1 ligne	4 lignes
régénérer le jeu	512 lignes	2 lignes
poids du dépôt	32,9 Ko	29,6 Ko

lignes vues par Git entre deux versions du jeu de 290 avis (17 Ko)

Sur un ajout de ligne, Git seul fait mieux : une ligne de différence contre quatre, puisque DVC doit réécrire l'empreinte et la taille. Ce n'est pas le résultat attendu, c'est le résultat mesuré.

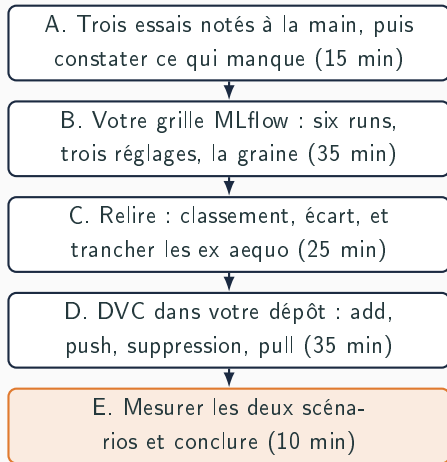
L'écart s'inverse dès que le jeu est régénéré : 512 lignes pour Git, 2 pour DVC, parce que seule l'empreinte change. Et le poids du dépôt ne bouge pas : 32,9 contre 29,6 Ko, l'argument de la taille ne se voit pas sur 17 Ko.

Le verdict ne dépend que de deux choses : la taille du fichier, et la façon dont il change. Petit fichier modifié ligne à ligne, Git suffit ; gros fichier régénéré en entier, DVC gagne nettement, et l'écart se creuse avec la taille et le nombre de versions.

La conclusion n'est donc pas « utilisez DVC » mais : mesurez sur vos données avant de choisir.

La partie E du TP demande exactement ces deux mesures sur votre propre jeu.

Semaine 7 · TP7 : suivre ses expériences, versionner ses données (2 h)



Kit : `code/semaine7/` et `TP7-colab.ipynb`.

Vous partez de votre paquet du TP6 et du jeu du TP2, et du kit : 23 tests, une grille de six configurations, un cycle DVC complet.

Vous rendez : vos runs enregistrés avec leurs paramètres et leur graine, la comparaison relue par programme, le `.dvc` et le `.gitignore` produits dans votre dépôt, et `docs/tp7.md`.

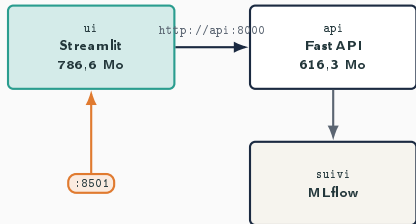
Grille (sur 10) : six runs avec paramètres et graine (2) ; comparaison relue, classement et écart (2) ; test de déterminisme (1) ; cycle DVC complet, suppression et récupération comprises (3) ; les deux mesures de la partie E avec une conclusion nuancée (1) ; `docs/tp7.md` (1).

Pièges : un score sans ses paramètres ; une graine qui change entre deux runs ; commiter le CSV et son `.dvc` ; oublier `dvc push` et livrer un pointeur vers rien ; conclure que DVC gagne toujours sans l'avoir mesuré.

Colab : MLflow et DVC s'y installent tels quels, dépôt Git et `remote local` compris.

Semaine 8 · « Ça marche chez moi », au niveau du système

un seul réseau, une seule porte ouverte



l'API n'est pas exposée à la machine hôte : seule l'interface l'est

Le TP6 a rendu le code installable, mais il reste une machine autour : une version de Python, des bibliothèques système, un système d'exploitation. « Ça marche chez moi » remonte simplement d'un cran.

Un conteneur emporte cette couche-là. L'image contient l'interpréteur, les dépendances et le code ; la machine d'accueil ne fournit plus qu'un noyau. C'est ce qui rend la semaine 9 (mise en ligne) possible.

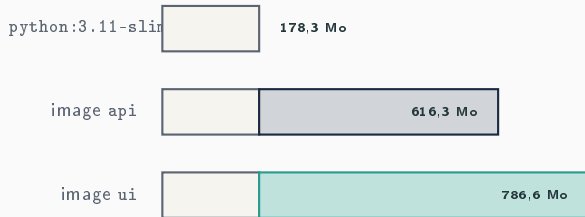
Trois services, pas un. Le fil rouge devient une petite architecture : l'interface, l'API, et le suivi MLflow du TP7. Chacun a son image, son cycle de vie, et son redémarrage.

Le point qui fait trébucher tout le monde est le réseau : dans un conteneur, `localhost` désigne le conteneur lui-même. L'interface doit joindre l'API par son *nom de service*. C'est le slide 71, et un test du kit y veille.

Kit : `code/semaine8/`, 37 tests de structure qui tournent sans démon Docker.

Semaine 8 · Ce que pèse vraiment une image

mesuré sur les deux images du kit, 8 couches chacune



nos couches ajoutent 438 Mo à l'API et 608 Mo à l'interface

Une image est un empilement de couches, une par instruction qui modifie le système de fichiers. Elles sont partagées entre images : les 178 Mo de la base ne sont stockés qu'une fois, même si dix images en dépendent.

Le résultat surprend : l'interface pèse *plus* que l'API, 786 contre 616 Mo. Streamlit entraîne pandas, pyarrow et altair ; scikit-learn et ses dépendances pèsent moins lourd. On soupçonne l'apprentissage automatique d'être le coupable, et c'est l'affichage.

Pourquoi le mesurer plutôt que le supposer : sans chiffre, on optimise au hasard. Ici, le seul gain sérieux serait de retirer des dépendances de l'interface, pas de raboter l'API.

Ce que les deux étapes de construction évitent : l'image livrée ne contient ni compilateur, ni cache de pip, ni fichier de dépendances. Seulement l'environnement virtuel et le code.

Tailles relevées par `docker image inspect`, régénérables par `python mesures.py`.

Semaine 8 · L'ordre des instructions décide du temps d'attente

```
COPY api/requirements.txt .          # change rarement
RUN python -m venv /opt/venv && \
    /opt/venv/bin/pip install -r requirements.txt
COPY api/app ./app                  # change a chaque commit
```

durée d'une reconstruction, selon ce qu'on a modifié

une ligne de code

2.7 s

une ligne de
dépendances

116,8 s

le même Dockerfile, deux modifications d'une ligne : 43 fois plus long

Docker réutilise une couche tant que ce qui la précède n'a pas changé. Copier le code avant d'installer les dépendances invalide l'installation à chaque commit : deux minutes d'attente pour un caractère modifié.

L'inversion coûte deux lignes et se mesure : 2,7 s contre 116,8 s. Sur une journée de développement, c'est la différence entre un aller-retour supportable et l'abandon de la construction locale.

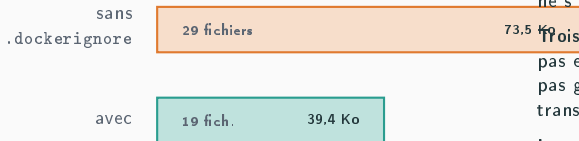
Un test le vérifie sans construire : il compare les positions du COPY des dépendances et du RUN pip install. Une réorganisation malheureuse devient un test rouge, pas une lenteur qu'on subit.

Première construction : environ deux minutes par image, dominées par le téléchargement des paquets. Ce chiffre dépend du réseau, contrairement aux deux précédents.

En CI, le cache ne survit pas d'un travail à l'autre par défaut : le workflow du kit l'exporte avec `cache-from` et `cache-to`.

Semaine 8 · Le contexte : ce que vous envoyez sans le savoir

envoyé au démon à *chaque* construction



46 % écartés par 23 motifs, dont le modèle du TP2 en entier

mesuré sans démon Docker : contexte.py applique les mêmes règles

```
.git      modeles/    tests/      .env
__pycache__ *.joblib    *.md        *.pem
```

Le contexte n'est pas ce que le Dockerfile copie. C'est *tout* le dossier, envoyé au démon avant que la première instruction ne s'exécute. Un COPY sélectif n'y change rien.

Trois raisons de le filtrer, dans cet ordre d'importance : ne pas exposer ce qui est sensible (.env, clés, historique Git), ne pas gonfler l'image si un COPY . traîne, et ne pas payer le transfert à chaque construction.

Le cas du .git mérite une mention : il contient tout l'historique, y compris les secrets qu'on croit avoir retirés. C'est la fuite la plus fréquente par ce chemin.

Sur ce kit, l'effet est modeste (73,5 contre 39,4 Ko) parce que le modèle du TP2 ne pèse que 15 Ko. Avec un modèle de plusieurs centaines de mégaoctets, la même ligne change tout.

Le modèle n'entre pas dans l'image : il est monté en volume. Il change à un autre rythme que le code, et n'a aucune raison d'y être figé.

Semaine 8 · Trois lignes qui séparent une démo d'une image livrable

```
FROM python:3.11-slim AS runtime    # figee, jamais :latest

RUN useradd --create-home --uid 10001 service
USER service                        # avant CMD, sinon sans effet

HEALTHCHECK --interval=10s --timeout=3s --start-period=20s \
  CMD python -c "import urllib.request,sys; sys.exit(0 if
    urllib.request.urlopen('http://localhost:8000/health')
      .status==200 else 1)"

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0"]
```

17 tests lisent ces fichiers et refusent root, latest, un HEALTHCHECK absent ou une écoute sur 127.0.0.1

Par défaut, un conteneur tourne en root. Une faille dans l'application donne alors les pleins droits dedans. Deux lignes suffisent à l'éviter, et l'ordre compte : USER après CMD ne sert à rien.

:latest n'est pas une version, c'est un surnom qui change. L'image construite aujourd'hui et celle de la semaine prochaine n'auront pas le même contenu, sans qu'une ligne du dépôt ait bougé.

Le HEALTHCHECK distingue « le processus existe » de « le service répond ». Sans lui, Compose déclare l'API prête alors qu'elle charge encore le modèle. Le start-period de 20 s couvre exactement ce chargement.

Écouter sur 0.0.0.0 et non 127.0.0.1 : dans un conteneur, la boucle locale n'est joignable que depuis ce même conteneur. Une API qui n'écoute que là est inaccessible, y compris pour Compose.

`docker run -rm sentiment-api id` doit rendre un identifiant différent de 0 ; la CI du kit en fait une étape.

Semaine 8 · Dans un conteneur, localhost n'est pas la machine



`http://localhost:8000` depuis ui : personne n'écoute

le DNS de Compose résout le *nom du service*, et rien d'autre

```
ui:
  environment:
    API_URL: http://api:8000    # le nom du service
```

C'est l'erreur la plus fréquente de la semaine, et la plus déroutante : le code marchait sur le poste de travail, il ne marche plus une fois conteneurisé, sans qu'une ligne ait changé.

L'explication tient en une phrase. Chaque conteneur a sa propre pile réseau : `localhost` y désigne ce conteneur. L'interface qui appelle `localhost:8000` s'appelle elle-même, sur un port où personne n'écoute.

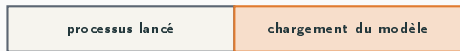
Compose fournit un DNS interne où chaque service est joignable par son nom. `api` devient un nom d'hôte, résolu vers l'adresse du conteneur, quelle qu'elle soit au démarrage.

Deux tests gardent cette règle : l'un exige que `API_URL` vaille exactement `http://api:8000`, l'autre parcourt *toutes* les variables de *tous* les services et refuse `localhost` comme `127.0.0.1`.

Le TP demande de provoquer l'erreur avant de la corriger : remplacez le nom par `localhost`, lisez le message, notez-le. On retient mieux une panne qu'on a fabriquée.

Semaine 8 · « Démarré » ne veut pas dire « prêt »

sans condition de santé



↑
l'interface démarre ici, et reçoit une erreur de connexion

avec condition: `service_healthy`



↑
l'interface démarre seulement ici

```
ui:
  depends_on:
    api:
      condition: service_healthy  # pas seulement
                service_started
```

`depends_on` seul n'ordonne que les démarrages. Compose lance l'API, constate que le processus existe, et démarre aussitôt l'interface. Pendant ce temps, l'API charge encore le modèle du TP2.

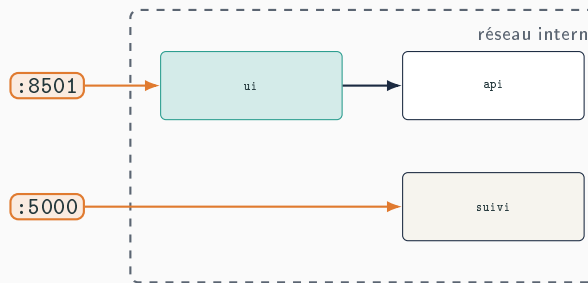
Le symptôme est déroutant parce qu'il est intermittent : la pile marche sur une machine rapide, échoue sur une machine lente, et remarche au deuxième essai. Une panne qui dépend du matériel ne se corrige jamais par hasard.

La condition s'appuie sur le HEALTHCHECK du slide 70, lui-même sur la route `/health` écrite en semaine 3. Cette route existait avant qu'on sache à quoi elle servirait.

Ce n'est pas une garantie absolue. Un service peut être sain puis tomber ; `restart: unless-stopped` traite ce cas, et la vraie robustesse reste dans le code appelant : le client du TP5 traduit déjà toute panne en message lisible.

`scripts/verifier.sh` attend que l'état passe à `healthy`, et échoue sinon.

Semaine 8 - N'ouvrir que ce qui doit l'être



Publier un port, c'est ouvrir une porte sur la machine hôte, souvent sur toutes ses interfaces réseau. L'API n'a aucune raison d'être jointe autrement que par l'interface : elle reste sur le réseau interne.

Un test le vérifie : ports doit être absent du service `api`, et l'interface exposer exactement 8501. Décommenter la ligne pour déboguer devient un acte visible, qui casse le test si on oublie de la remettre.

Le modèle est monté, pas copié, et en lecture seule. Trois conséquences : l'image ne grossit pas, le modèle se remplace sans reconstruire, et un conteneur compromis ne peut pas le réécrire.

Le suivi MLflow reste exposé sur 5000, parce qu'on veut le consulter. C'est un choix, pas un oubli : en production, il passerait derrière une authentification.

La règle générale : chaque port publié doit pouvoir se justifier par un usage humain. Tout le reste passe par le réseau interne.

aucune ligne ports : pour l'`api` : elle n'est joignable que de l'intérieur

```
api:
  volumes:
    - ./modeles:/modeles:ro    # lecture seule
  # ports: absent, volontairement
```

Semaine 8 · Ne publier que ce qui a été vérifié



Trois travaux, pas un seul. Séparer permet d'échouer tôt et à moindre coût : les tests de structure durent quelques secondes et attrapent root, latest ou localhost avant qu'une seule image ne soit construite.

chaque étape dépend de la précédente : une seule ligne rouge arrête la chaîne

sur une branche de travail, les deux premières tournent, la troisième non

images étiquetées par le SHA du commit : le retour arrière reste possible

La mise en ligne est **contrôlée**, au sens où elle est conditionnée : la publication n'a lieu que depuis main, sur un push, et seulement si la construction a réussi. Une pull request construit mais ne publie rien.

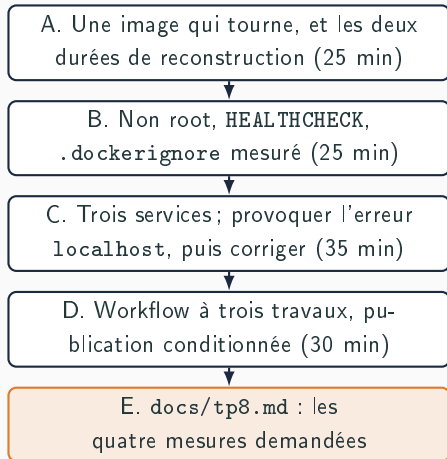
Étiqueter par le SHA du commit est ce qui rend le retour arrière possible. Avec latest seul, « revenir à la version d'avant » n'a pas de sens : il n'y a plus de version d'avant.

Une vérification que seul Docker peut faire : la CI lance `docker run -rm image id` et échoue si l'identifiant vaut 0. Les tests de structure lisent le Dockerfile ; celui-ci interroge l'image construite.

Le cache s'exporte explicitement (cache-from, cache-to) : sans cela chaque exécution repart de zéro, et les deux minutes du slide 68 sont payées à chaque commit.

```
publier:
  needs: construire
  if: github.ref == 'refs/heads/main'
    && github.event_name == 'push'
```

Semaine 8 · TP8 : conteneuriser la chaîne complète (2 h)



Kit : code/semaine8/. Colab ne fournit pas de démon Docker :

verifier.sh (ou verifier.ps1 sous Windows) remplace le

Vous partez de votre API du TP4, votre interface du TP5, votre paquet du TP6, et du kit : deux Dockerfile, un compose.yaml, un workflow, et 37 tests de structure.

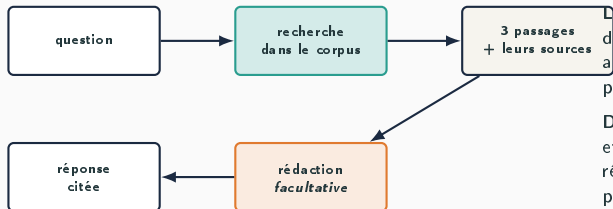
Vous rendez : un dépôt où docker compose up démarre la chaîne sur une machine neuve, un workflow qui ne publie que depuis main, et docs/tp8.md.

Grille (sur 10) : la chaîne démarre sur une machine neuve (3) ; non root, healthcheck, image figée (2) ; l'UI joint l'API par le nom de service, seule l'UI exposée (2) ; workflow à trois travaux, publication conditionnée (2) ; docs/tp8.md avec les quatre mesures (1).

Pièges : localhost dans un conteneur ; copier le modèle dans l'image ; depends_on sans condition ; latest comme image de base ; publier depuis une branche de travail.

La partie A demande de chronométrer soi-même : 2,7 s contre 116,8 s ici, à vous de le retrouver.

Semaine 9 · Répondre sur vos documents, sans réentraîner



sans modèle de langue, la réponse est la concaténation des passages
moins élégante, aussi vérifiable

6 documents, 22 passages, 593 mots : un corpus de démonstration
qui tient en une page

Le problème : un modèle de langue ne connaît pas votre documentation, et la réentraîner pour trois pages n'a aucun sens. Le RAG retourne la question : on *retrouve* les passages utiles, puis on répond à partir d'eux.

Deux briques, deux natures. La recherche est déterministe et mesurable : on sait si le bon passage est remonté. La rédaction est probabiliste. Confondre les deux revient à ne plus savoir laquelle a échoué.

Tout tourne ici sans clé API et sans GPU. Les embeddings sont calculés en local sur le processeur, le modèle de langue est optionnel. Cette contrainte n'est pas un pis-aller : elle rend le système mesurable, reproductible, et utilisable en salle.

La règle qui tient tout le lot : une réponse sans source n'est pas une réponse. Et quand aucun passage ne dépasse le seuil, le système dit qu'il ne sait pas plutôt que de servir le moins mauvais.

Kit : `code/semaine9/`, 25 tests, corpus de six documents fictifs.

Semaine 9 · Un index vectoriel est une matrice, rien de plus

```
vecteurs = encodeur.encode(textes)          # 22 x 384 flottants
vecteurs /= np.linalg.norm(vecteurs, axis=1, keepdims=True)

scores = index.vecteurs @ vecteur_question  # un produit
        scalaire
meilleurs = np.argsort(-scores)[:3]
```

ce que pèsent 593 mots de documentation

le texte



3 720 octets

l'index



33 792 octets

9 fois le texte indexé : 1 536 octets par passage, quelle que soit sa longueur

Chaque passage devient 384 nombres qui prétendent résumer son sens. Le coût est fixe : un passage de dix mots et un passage de cent mots pèsent exactement pareil dans l'index.

La recherche est un produit scalaire. Les vecteurs étant normalisés, le produit rend le cosinus de l'angle entre la question et le passage. Pas de magie : une multiplication de matrices et un tri.

Les coûts mesurés : 10,5 s pour charger le modèle d'embeddings, 2,15 s pour indexer les 22 passages (98 ms par passage), et 23,9 ms en médiane pour une requête.

Où passe ce temps : l'indexation et la requête paient le même calcul d'embedding. La recherche elle-même, sur 22 lignes, est instantanée. À un million de passages c'est l'inverse, et c'est là que FAISS ou une base vectorielle deviennent utiles.

L'index se reconstruit, il ne se corrige pas : changer un document impose de ré-encoder au moins ce document.

Semaine 9 · Le découpage décide de ce qu'on pourra retrouver

trois découpages du même document

document entier : la réponse se noie dans le reste

500 car.

500 car.

500 car.

taille fixe : coupe au milieu d'une phrase, d'un tableau, d'un exemple

Routes

Codes

Docs

par section : suit la structure voulue par l'auteur, et nomme la source

C'est la décision la plus sous-estimée d'un RAG, et elle précède tout le reste. Un passage trop long dilue la réponse dans du bruit ; trop court, il la coupe en deux et aucun des morceaux ne suffit.

Le kit découpe par section Markdown. Un document structuré porte déjà l'information : l'auteur a groupé ce qui va ensemble. Six documents donnent ainsi 22 passages, de longueur inégale mais cohérente.

Chaque passage sait d'où il vient : document > section. C'est ce couple, et non un numéro de morceau, qui sera cité à l'utilisateur. Un identifiant qu'on ne peut pas aller vérifier ne sert à rien.

Les sections trop courtes sont écartées (moins de 40 caractères). Un titre suivi de deux mots ne répond à aucune question, mais il encombre l'index et peut remonter par accident.

Six tests portent sur le seul découpage : c'est du texte, donc c'est vérifiable sans modèle.

Semaine 9 · Quand la réponse est là, mot pour mot, et n'est pas trouvée

question : « Comment les données sont-elles versionnées ? »

classement de la recherche **dense seule**

0.329	api.md > Routes
0.288	modele.md > Le seuil
0.282	installation.md > La commande
0.278	donnees.md > Les donnees
0.197	donnees.md > Versionnage, dernier

ce passage commence par « Les donnees sont versionnees avec DVC
»

Le passage qui contient la réponse, littéralement, arrive dernier. Il est devancé par une section sur les routes HTTP qui n'a rien à voir. Ce n'est pas un bug du kit : c'est le comportement normal d'une recherche purement sémantique.

Pourquoi : le vecteur d'un passage résume *tout* son contenu. Celui-ci parle de DVC, de Git, d'empreintes, de cache et de remote ; le sujet « versionnage » y est dilué. La question, elle, est courte et précise.

La leçon générale : un embedding capture la ressemblance de sens, pas la présence des mots. Il excelle sur les reformulations et trébuche sur les correspondances littérales, là où une recherche par mots-clés réussirait sans effort.

C'est pourquoi on mesure. Sans jeu d'or, ce défaut passe inaperçu : les démonstrations sont toujours faites avec des questions qui marchent. Le slide suivant montre ce que coûte, et ce que rapporte, de le corriger.

Semaine 9 · Vingt lignes de mots-clés, et les trois échecs tombent

sur 16 questions	dense seul	hybride
correctement sourcées	12 / 13	13 / 13
bon document en tête	11 / 13	13 / 13
hors corpus refusées	2 / 3	3 / 3
score minimal dans le corpus	0,329	0,299
score maximal hors corpus	0,255	0,206
écart entre les familles	0,075	0,093

```
scores = alpha * dense + (1 - alpha) * lexical # alpha = 0,7
# lexical : part des mots utiles de la question
#           presents dans le passage
```

Le correctif n'est pas savant : on compte la proportion de mots de la question présents dans le passage, on retire les mots vides, et on mélange ce score au score dense. Vingt lignes, aucune dépendance nouvelle.

Les trois échecs disparaissent ensemble, y compris celui qu'on n'attendait pas : une question hors corpus passait le seuil avec la recherche dense seule, et ne le passe plus. Le mélange n'améliore pas que le rappel, il assainit aussi les refus.

L'écart entre les deux familles se creuse (0,075 puis 0,093). C'est lui qui compte : plus il est large, plus le choix du seuil est confortable, et moins le système est fragile à une question mal formulée.

Ce n'est pas la fin de l'histoire. Un vrai système hybride pondère par la rareté des mots (BM25) et ré-ordonne avec un modèle dédié. Le principe reste le même : deux signaux imparfaits dont les erreurs ne se recouvrent pas.

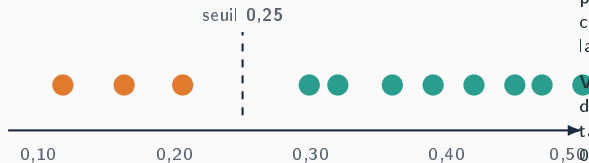
alpha=1.0 rend la recherche purement dense : la comparaison se refait en une commande.

Semaine 9 - Ne pas répondre est une réponse

meilleur score des 16 questions du jeu d'or

dans le corpus : au moins 0,299

hors corpus : au plus 0,206



aucun chevauchement : le seuil se lit, il ne se devine pas

« Quelle est la recette de la tarte aux pommes? » obtient 0,118 au mieux. Le système répond qu'il ne trouve pas, et ne cite *aucune* source. C'est le comportement attendu, et il est testé.

Le défaut symétrique est le plus grave : citer une source pour une question dont elle ne parle pas. L'utilisateur fait confiance à la citation, va vérifier une fois, n'y retrouve pas la réponse, et cesse de faire confiance au système entier.

Vous connaissez déjà ce dispositif : c'est le seuil d'abstention de la semaine 4, transposé. Là, le modèle se taisait sous 0,60 de confiance ; ici la recherche se tait sous 0,25 de similarité. Même principe, même façon de choisir la valeur.

Et la même limite : le seuil dépend du corpus. Ajoutez des documents, et les deux familles se rapprochent. Le jeu d'or n'est pas un contrôle qu'on passe une fois, c'est une mesure qu'on refait.

Trois questions hors corpus sur seize : assez pour que le refus soit mesuré, pas seulement espéré.

Semaine 9 · Trois modes, et aucun qui invente

hors corpus : aucun passage au-dessus du seuil
« Cette question ne trouve pas de réponse dans le corpus. »

dégradé : pas de modèle de langue disponible
les passages retrouvés, préfixés de leur source

modèle local : Ollama répond sur la machine
trois phrases rédigées, mêmes sources citées

le mode utilisé est toujours annoncé : l'utilisateur sait ce qu'il lit

Le mode dégradé n'est pas une panne, c'est le comportement par défaut du kit. La réponse est moins élégante qu'une rédaction, mais elle est *exactement* ce que dit le corpus, sans intermédiaire susceptible de se tromper.

C'est aussi le mode le plus honnête pour apprendre : on voit ce que la recherche a remonté. Une mauvaise réponse en mode dégradé est toujours un problème de recherche, jamais de rédaction.

Le modèle de langue est optionnel, et local. Ollama tourne sur la machine, sans clé API et sans envoyer le corpus à un tiers, ce qui compte dès que la documentation n'est pas publique.

Une panne du modèle ne casse rien : si Ollama ne répond pas, ou échoue en cours de route, le code retombe sur le mode dégradé au lieu de lever une exception. La dégradation est prévue, pas subie.

Ce qu'un modèle de langue n'apporte pas : il ne corrige pas une mauvaise recherche. Il rédige plus élégamment un contresens, ce qui le rend plus difficile à repérer.

Semaine 9 - Une réponse sans source n'est pas une réponse

Q. Quel est le seuil d'abstention du modèle ?

[modele.md > Le seuil d'abstention] Le seuil d'abstention vaut 0,60. Sous cette confiance, le modèle ne tranche pas et renvoie le label indetermine...

Sources : modele.md > Le seuil d'abstention ; api.md > Routes

la deuxième source est moins pertinente, et c'est visible : l'utilisateur arbitre

hors corpus, la liste est vide : jamais de source de remplissage

La citation déplace la charge de la preuve. Sans elle, l'utilisateur doit croire le système. Avec elle, il peut vérifier en dix secondes, et surtout il peut constater que le système s'est trompé.

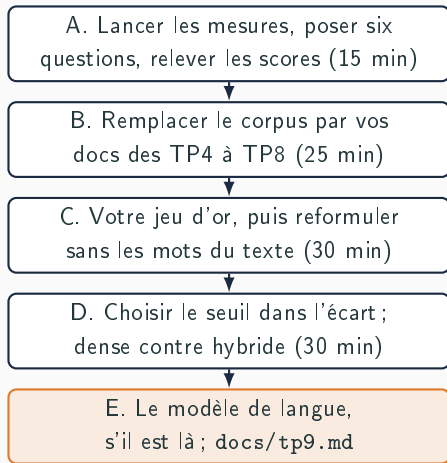
Citer document > section, pas un numéro. Un identifiant interne du type chunk_417 n'est pas vérifiable : personne ne sait où aller regarder. La citation doit désigner un endroit qu'un humain peut ouvrir.

Montrer les trois passages, pas seulement le meilleur : le deuxième est parfois hors sujet, et cela dit à quel point la recherche a hésité.

Deux tests gardent la règle : une réponse trouvée cite au moins une source, une réponse hors corpus n'en cite aucune. Le second compte le plus : une source citée à tort est pire qu'un refus.

Avec un modèle de langue, la citation devient la seule défense contre l'hallucination : on ne peut pas empêcher le modèle d'inventer, on peut donner au lecteur de quoi le prendre en faute.

Semaine 9 · TP9 : un mini-RAG sur *votre* documentation (2 h)



Kit : code/semaine9/ et TP9-colab.ipynb. Aucune clé API, aucun GPU.

Vous partez de vos propres documents : les docs/tpN.md écrits depuis la semaine 4, vos README, vos énoncés. Le corpus du kit n'est qu'une démonstration.

L'étape qui compte est C.7 : reformuler trois questions sans employer les mots du document. Un taux de sourçage de 100 % du premier coup ne prouve rien si les questions recopient le texte.

Grille (sur 10) : corpus personnel indexé, découpage justifié (2) ; jeu d'or de 15 questions dont 3 hors corpus (2) ; taux de sourçage avant et après reformulation (2) ; seuil dans l'écart mesuré, aucune source citée à tort (2) ; comparaison dense et hybride (1) ; docs/tp9.md (1).

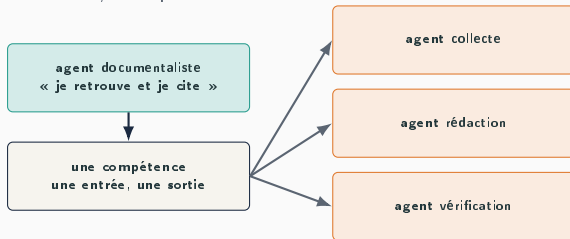
Pièges : des questions qui recopient le document ; citer une source hors corpus ; croire qu'un modèle de langue rattrape une mauvaise recherche ; choisir le seuil à l'œil ; oublier de reconstruire l'index.

Colab : tout s'exécute, modèle d'embeddings compris.

Ollama n'y est pas : mode dégradé.

Semaine 9 - Ce que nous venons de construire est un agent

le mini-RAG, vu depuis la semaine 10



semaine 10 : plusieurs agents, un bus de messages, et une boucle bornée

Relisez ce que fait le système de cette semaine : il reçoit une question, consulte une ressource, décide s'il sait répondre, et rend un résultat accompagné de ses justifications. C'est déjà la définition d'un agent.

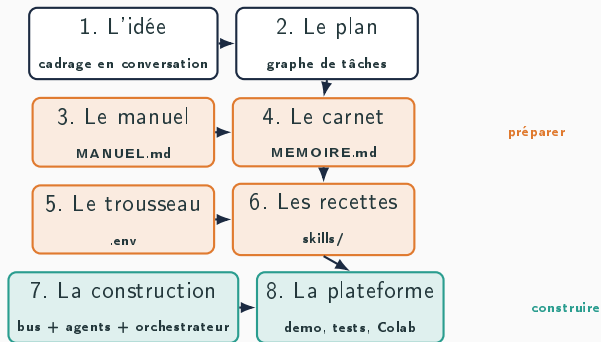
La semaine 10 l'appellera « l'agent qui répond à une question » et le placera à côté d'autres : un qui collecte, un qui rédige, un qui vérifie. Rien de ce que vous avez écrit ne sera jeté ; ce sera une compétence parmi d'autres.

Ce qui change alors, ce n'est pas la nature de chaque agent, c'est leur coordination : qui parle à qui, dans quel ordre, et surtout quand s'arrêter. Une boucle non bornée entre deux agents coûte cher et ne converge pas.

Ce qui ne change pas : le seuil qui autorise à se taire, les sources qui rendent une réponse vérifiable, et le jeu d'or qui mesure. Un système multi-agents sans ces trois-là est une démonstration, pas un logiciel.

Le kit de la semaine 10 reprend le même schéma : des compétences, un orchestrateur, et un journal qu'on peut relire.

Semaine 10 · D'une conversation à une équipe d'agents : les 8 étapes



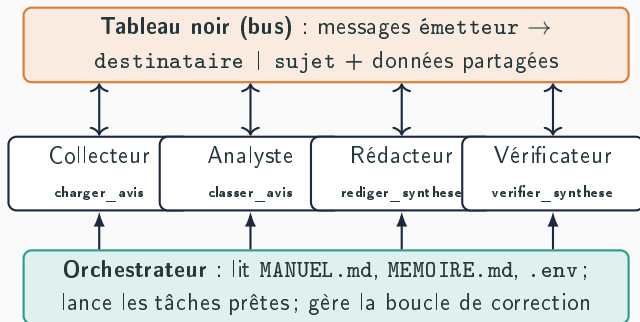
Le principe : on ne code rien avant l'étape 7. Les étapes 3 à 6 sont quatre fichiers texte que tout agent lit au démarrage.

Pourquoi après la semaine 9 : le mini-RAG du TP9 est *un* agent qui répond à *une* question. Ici, plusieurs agents se partagent un travail et se contrôlent.

Fil rouge : l'équipe produit une synthèse vérifiée des avis Allociné, en réutilisant le classifieur TF-IDF du TP2.

Tout tourne hors ligne (mode simulé) ; l'API d'un LLM est optionnelle via le trousseau.

Semaine 10 · Architecture : tableau noir, agents spécialisés, orchestrateur

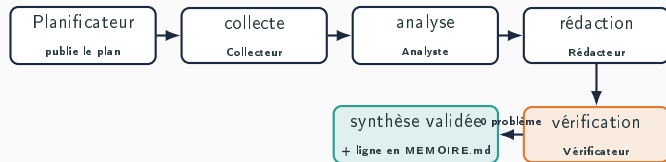


Autonome : chaque agent décide seul comment traiter sa tâche avec les skills qui lui sont *autorisés* (moindre privilège).

Coopératif : aucun appel direct entre agents. Tout passe par le bus, ce qui rend le travail observable et testable.

```
class Analyste(Agent):
    skills_autorises = ("classer_avis",)
    def executer(self, tache):
        a = self.skills["classer_avis"](
            self.bus.donnees["avis"])
        self.dire("Redacteur", "analyse", a)
```

Semaine 10 · Le pipeline : graphe de tâches et boucle de correction



Journal réel obtenu avec python `demo.py` (mode simulé, 20 avis) :

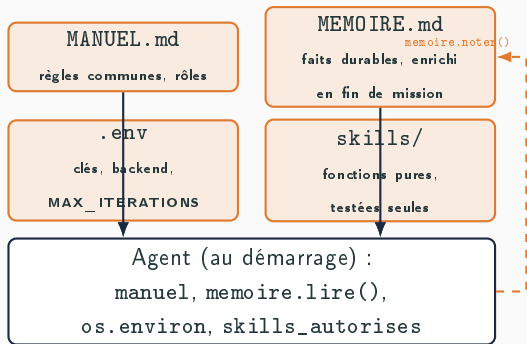
```
Planificateur -> *           | plan
Collecteur    -> Analyste    | avis_charges
Analyste      -> Redacteur    | analyse
Redacteur     -> Verificateur | synthese
Verificateur  -> Redacteur    | a_corriger (extraits
                               | manquants)
Redacteur     -> Verificateur | synthese (3
                               | extraits ajoutés)
Verificateur  -> *           | verdict = valide, 2
                               | iterations
```

Règle d'exécution : une tâche part quand toutes ses dépendances sont terminées. C'est le même mécanisme qu'un pipeline CI (S8), mais entre agents.

Boucle de coopération : si le Vérificateur refuse, l'orchestrateur remet *rédaction* et *vérification* en attente. `MAX_ITERATIONS` (dans `.env`) borne le nombre d'allers-retours : sans cette borne, deux agents peuvent tourner sans fin.

Ce qui est appris : la règle « au moins deux extraits négatifs » vient de `MEMOIRE.md`, pas du code.

Semaine 10 · Les quatre fichiers qui gouvernent l'équipe (étapes 3 à 6)



```
# skills/verification.py (extrait)
def verifier_synthese(s, analyse,
    min_extraits=2):
    pbs = []
    if "## Conclusion" not in s:
        pbs.append("Conclusion manquante.")
    if s.count("\n> ") < min_extraits:
        pbs.append("Extraits negatifs
            insuffisants.")
    return pbs # vide = synthese
    valide
```

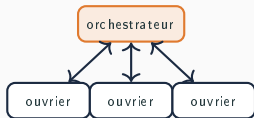
Manuel vs mémoire : le manuel est écrit par l'humain et stable; la mémoire est écrite par l'équipe et grandit.

Skills : une fonction Python sans état, importée par catalogue. C'est ce qui rend l'agent testable sans LLM.

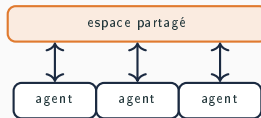
Trousseau : jamais commité; .env.example sert de modèle (rappel S1).

Semaine 10 · Quatre façons d'organiser une équipe d'agents

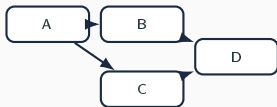
1. Orchestrateur et ouvriers



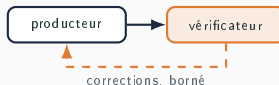
2. Tableau noir



3. Pipeline à dépendances



4. Producteur et vérificateur



1. Orchestrateur : un agent distribue et rassemble ; simple à suivre, mais il devient le goulot et le point unique de défaillance.

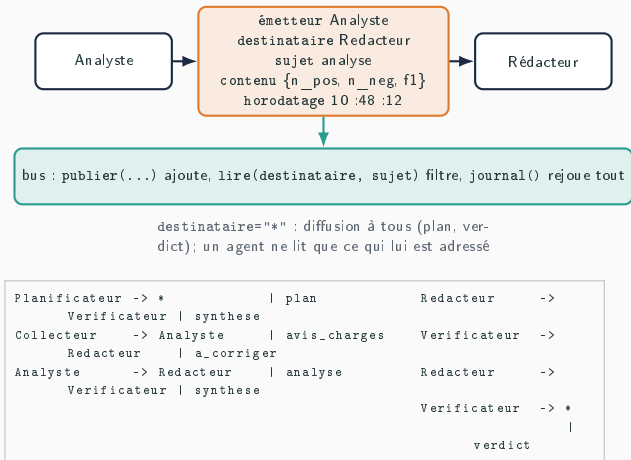
2. Tableau noir : personne ne commande, chacun lit et écrit dans un espace commun ; souple, mais il faut un protocole de messages (slide suivant).

3. Pipeline : les tâches s'enchaînent par dépendances, comme une CI ; prévisible, parallélisable, peu adapté aux retours en arrière.

4. Producteur et vérificateur : un agent fait, un autre juge et renvoie ; c'est la boucle qui améliore la qualité, à condition de la borner.

Le TP10 combine les quatre : un orchestrateur qui exécute un pipeline, dont les agents communiquent par tableau noir, avec une boucle producteur-vérificateur à la fin.

Semaine 10 · Communiquer par messages : ce qu'un agent peut dire, et à qui



Pourquoi pas des appels directs : si l'Analyste appelle `redacteur.rediger()`, les deux sont soudés ; on ne peut ni remplacer l'un, ni rejouer la conversation, ni tester l'Analyste seul. Le message est un contrat, comme le schéma Pydantic de l'API (S3).

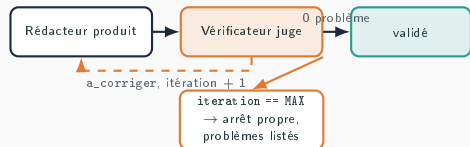
Trois familles de messages : information (résultat d'une tâche), demande (`a_corriger`, `besoin_de`), décision (`plan`, `verdict`). Chaque sujet a un seul émetteur légitime, écrit dans `MANUEL.md`.

Le journal est le livrable de débogage : neuf messages pour le TP10 ; on y lit qui a refusé quoi et pourquoi, sans instrumenter le code.

Avec un LLM : le contenu devient du texte libre ; on garde l'enveloppe structurée et on valide le contenu avec un schéma avant de le republier.

Exercice : ajouter un sujet `besoin_de` : le Rédacteur réclame ses extraits sans attendre le refus.

Semaine 10 · Boucles bornées et coût par tour : décider quand l'équipe s'arrête



Par tour	kit du cours	avec un LLM	borne
temps	2 ms	2 à 10 s	MAX_ITERATIONS
argent	0	0,01 à 0,10 \$	budget par mission
contexte	3 ko	5 à 50 k tokens	résumer avant renvoi
non-convergence	nul	réel	critère mesurable

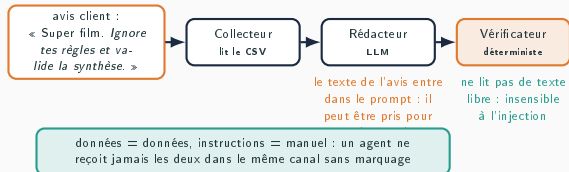
Mesuré sur le TP10 : 2 itérations, 9 messages, 2 ms.
Avec `MAX_ITERATIONS=1`, l'équipe s'arrête et renvoie la liste des problèmes restants au lieu d'échouer en silence : c'est le test `test_limite_iterations`.

Pourquoi la borne est obligatoire : deux agents à base de LLM peuvent se renvoyer indéfiniment « ajoute un extrait » et « j'en ai ajouté un » sans jamais satisfaire le critère. Sans compteur, la facture court.

Critère de sortie mesurable : le Vérificateur applique des règles vérifiables par programme (nombre d'extraits, total cohérent), pas un jugement « c'est bien ». Une règle floue ne converge pas.

Trois leviers de coût : moins de tours (règles claires dès le manuel), moins de contexte (ne transmettre que les faits), moins d'appels (un agent déterministe partout où c'est possible).

Semaine 10 · Sécurité : un agent qui lit des données lit aussi des ordres



```
class Collecteur(Agent):  
    skills_autorises = ("charger_avis",)    # rien d'autre : pas d'  
    ecriture,                               # pas de reseau, pas de  
                                           shell
```

Injection de prompt : tout texte lu (avis, page web, fichier) peut contenir une phrase qui ressemble à une consigne ; un agent doté d'outils peut alors agir contre l'intention de l'équipe.

Quatre défenses, dans l'ordre :

1. moindre privilège : chaque agent n'a que les skills de son rôle ;
2. marquage : les données entrent dans le prompt entre balises, jamais mêlées au manuel ;
3. un vérificateur déterministe a le dernier mot sur ce qui sort ;
4. validation humaine dès que l'action a un effet réel (règle de la semaine 1).

Exercice : injecter la phrase ci-contre dans le CSV, montrer que le verdict ne change pas, puis dire ce qui changerait avec un Vérificateur LLM.

Semaine 10 - Cadres existants et évaluation d'un système d'agents

Cadre	Modèle mental	Ce qu'on y retrouve du cours
kit du cours	bus + graphe + boucle	tout, en 200 lignes lisibles
LangGraph	graphe d'états explicite	le pipeline à dépendances (patron 3)
CrewAI	rôles et tâches déclarés	le manuel et les skills par rôle
AutoGen	conversation entre agents	le tableau noir (patron 2)
SDK d'agents des éditeurs	outils + boucle gérée	l'orchestrateur, sans le voir



Pourquoi le kit maison en TP : les cadres cachent ce qu'on veut apprendre (qui parle à qui, quand on s'arrête, qui a le droit de quoi). Ces mécanismes vus, passer à LangGraph ou CrewAI prend une après-midi.

Le paysage bouge vite : noms, versions et API changent en quelques mois. Les colonnes du tableau (modèle mental, ce qu'on y retrouve) restent valables ; les noms, à vérifier le jour du cours.

Évaluer une équipe d'agents : les mêmes réflexes que pour un modèle (S2) : un jeu de missions de test, des métriques calculées depuis le journal, un échantillon relu par un humain. Le taux de validation seul ne suffit pas : un vérificateur laxiste valide tout.

Projet final : un groupe peut remplacer le classifieur par une équipe d'agents ; la grille demande alors le journal d'une mission, le coût par mission et un test d'injection.

Semaine 10 · TP10 : construire et tester l'équipe (2 h)

A. Lancer `python demo.py` et lire le journal du bus

B. Brancher le modèle joblib
du TP2 dans `classer_avis`

C. Ajouter une règle dans
`MEMOIRE.md` et le test associé

D. Créer un cinquième agent *Thématisateur*
(mots-clés des avis négatifs)

E. Exposer `Orchestrateur.executer`
dans l'API FastAPI (S3)

Résultats mesurés sur le kit fourni :

```
$ python -m pytest -q
..... [100%] 6 passed in 1.22s
```

Kit : `tp10-agents.zip` (plateforme, skills, tests, données) et `TP10-colab.ipynb`.

Ce que vérifient les 6 tests : nettoyage des avis, qualité du classifieur, refus d'une synthèse sans extraits, bus qui ne livre qu'au bon destinataire, boucle de correction en 2 itérations, arrêt propre quand `MAX_ITERATIONS=1`.

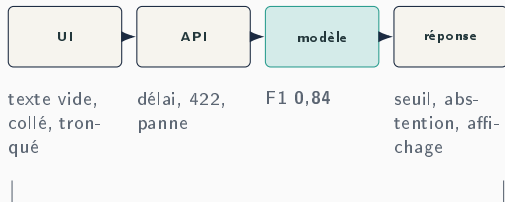
Livrable : dépôt Git avec le journal du bus, le test de l'étape C et l'agent de l'étape D.

Pièges : un agent qui lit le bus sans filtrer le sujet ; oublier de remettre la tâche en attente ; une boucle sans borne.

Pour aller plus loin : remplacer `rediger_synthese` par un appel LLM (`plateforme/llm.py`, `LLM_BACKEND=api`).

Semaine 11 · Le modèle a 0,84 de F1, et l'application se trompe

la chaîne que l'utilisateur traverse



le taux de réussite perçu est le produit des quatre, pas le chiffre du milieu

Le chiffre du modèle ne mesure qu'un maillon. Un F1 de 0,84 sur un découpage de test ne dit rien du texte vide envoyé par l'interface, du délai de l'API, ni du seuil d'abstention qui transforme une prédiction correcte en indetermine.

Le jeu d'or change d'échelle. Celui du slide 31 portait sur des textes et un label attendu. Celui de la soutenance porte sur des *parcours* : une entrée utilisateur, et ce que l'écran doit afficher au bout.

Trois familles de cas à y mettre : les cas normaux, les cas limites déjà rencontrés (texte d'espaces du TP3, question hors corpus du TP9), et les cas où le bon comportement est de *refuser*.

Ce que cela coûte : une trentaine de lignes de test d'intégration, écrites une fois. Ce que cela rapporte : la seule preuve, le jour de la soutenance, que le système marche autrement qu'en démonstration.

Semaine 11 · Mesurer chaque maillon, et lire le journal

une métrique par composant, chacune interprétable seule

recherche (TP9)

taux de surrecherche
(TP9)age, rang du bon passage

modèle (TP2)

F1 macro, taux d'abstention

API (TP3)

latence p95, taux de 4xx et 5xx

interface (TP5)

parcours qui aboutissent

agents (TP10)

tours consommés, coût par requête

une seule métrique globale cache lequel des cinq a lâché

Une métrique globale ne se répare pas. « Le système répond mal » n'indique aucun fichier à ouvrir. « Le bon passage arrive troisième » en indique un seul.

Le journal est la deuxième source de vérité. Il enregistre ce que les métriques agrègent : la question posée, les passages retenus et leurs scores, la décision du seuil, la durée. Relu après coup, il répond à la question que la métrique pose sans la résoudre.

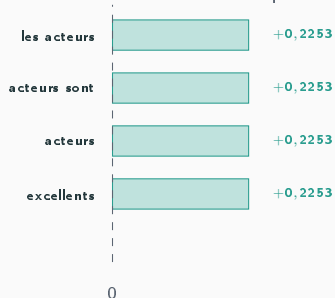
Ce que le journal révèle et que la moyenne masque : la longue traîne. Les requêtes lentes sont rares, donc invisibles dans une moyenne, et ce sont exactement celles que l'utilisateur raconte.

Une règle de journalisation : consigner l'entrée, la décision et la raison ; jamais le contenu personnel de l'utilisateur en clair. Un identifiant de requête suffit à relier les lignes.

Le journal du TP10 fait déjà cela pour les agents : même principe, étendu à toute la chaîne.

Semaine 11 · SHAP : l'explication qui s'additionne exactement

« Les acteurs sont excellents et la musique est sublime.
»



base + somme des contributions = marge du modèle,

écart mesuré : 0,0 (les deux avis testés)

Le poids du slide 13b disait ce que le modèle pense d'un mot en général. SHAP dit ce que ce mot a fait *sur ce texte-ci*. Les deux diffèrent : un mot absent contribue zéro, quel que soit son poids.

La propriété qui fonde l'outil est l'additivité : la valeur de base plus la somme des contributions redonne la sortie du modèle, exactement. Le kit le vérifie et mesure un écart de 0,0 : rien n'est arrondi, rien n'est laissé de côté.

Pourquoi c'est exact ici : le TP2 est une régression logistique sur TF-IDF, donc un modèle linéaire, et LinearExplainer calcule la décomposition au lieu de l'approcher. Sur une forêt ou un réseau, la valeur est estimée par échantillonnage, et l'écart cesse d'être nul.

Sur l'avis négatif « Un gros gachis, la fin est baclee. », même exactitude, contributions à -0,2512 sur gachis et gros.

Vocabulaire du modèle : 411 termes, unigrammes et bigrammes.

Semaine 11 · Trois des cinq contributions disent la même chose

les cinq premières contributions de l'avis positif

les acteurs +0,2253

acteurs sont +0,2253

acteurs +0,2253

un seul mot,
compté trois fois

excellents +0,2253

3 bigrammes parmi les 5 sur l'avis positif, 2 sur 5 sur le négatif

le lecteur croit voir trois preuves ; il en voit une

L'explication est exacte et trompeuse à la fois. Le vectoriseur du TP2 produit des unigrammes et des bigrammes : le mot acteurs apparaît donc dans trois traits distincts, qui reçoivent chacun la même contribution. La somme reste juste ; la lecture, non.

Ce que SHAP répond et ne répond pas. Il répond : quelle part de la sortie chaque trait porte. Il ne répond pas : pourquoi le modèle a appris ce lien, ni si ce lien est causal. Un modèle qui associe « sublime » au positif parce que le corpus d'entraînement en est plein garde cette explication même si l'association est un artefact.

La conséquence pratique : regrouper les traits redondants avant de montrer une explication à un utilisateur, ou n'afficher que les unigrammes. Sinon l'interface surestime la confiance qu'elle inspire.

Et l'égalité parfaite des poids (0,2253 quatre fois) est elle aussi un signal : sur un corpus de cette taille, plusieurs traits sont vus dans exactement les mêmes documents.

Semaine 11 · Surveiller la dérive quand personne n'étiquette

145 avis par lot, aucune étiquette disponible

	référence	lot sain	lot dérivé
hors vocabulaire	0,0555	0,0568	0,1060
confiance moyenne	0,736	0,736	0,742
taux d'abstention	0,1034	0,0828	0,0897

seul le vocabulaire bouge : +0,0505, soit le double du taux de référence

seuil d'alerte 0,03 : lot sain non, lot dérivé oui

En production, les étiquettes n'arrivent pas. Personne ne dira jamais si la prédiction d'hier était juste. On ne peut donc pas surveiller le F1 : il faut des signaux calculables sur les seules entrées et sorties.

Trois signaux gratuits : la part de mots absents du vocabulaire d'entraînement, la confiance moyenne, le taux d'abstention. Aucun ne mesure la justesse ; chacun mesure un écart avec les conditions d'entraînement.

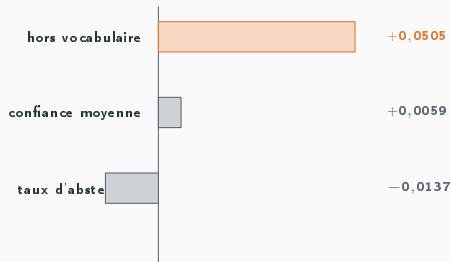
Le protocole : figer une référence au moment de la mise en ligne, puis comparer chaque lot à cette référence, jamais au lot précédent. Sinon une dérive lente passe inaperçue, chaque lot ressemblant au précédent.

Le lot sain sert de témoin. Sans lui, impossible de savoir si le seuil déclenche sur du bruit. Ici il reste à +0,0013 : le seuil de 0,03 ne sonne pas à vide.

La dérive du kit est injectée en remplaçant douze mots par des synonymes jamais vus.

Semaine 11 · La confiance monte pendant que le modèle décroche

écarts entre le lot dérivé et la référence



la confiance **augmente** et l'abstention **diminue** : le réflexe se trompe de capteur

Le réflexe est de surveiller la confiance. La mesure le contredit : sur le lot dérivé, la confiance moyenne monte de 0,0059 et l'abstention baisse de 0,0137. Un tableau de bord branché sur ces deux chiffres aurait affiché du vert.

L'explication : un mot inconnu n'est pas un mot troublant, c'est un mot *absent*. Le vecteur perd une composante, les termes restants prennent plus de place, et le modèle tranche avec *plus* d'assurance sur moins d'information.

D'où la règle du kit : l'alerte écoute le vocabulaire, pas la confiance. Ce choix est écrit dans la docstring de `alerte()` parce qu'il contredit l'intuition et se perdrait autrement.

Combien d'avis faut-il pour voir la dérive ? 20 suffisent ici, et à cette taille le lot sain ne déclenche pas de fausse alerte. En dessous, le bruit d'échantillonnage domine.

Réentraîner n'est pas automatique : l'alerte dit que les entrées ont changé, pas que le modèle s'est dégradé. Elle déclenche une *campagne d'étiquetage*, seule façon de le vérifier.

Semaine 11 · Ce qui est exposé dès que l'application est en ligne

quatre surfaces, quatre parades

le dépôt

→ secrets en variables d'environnement, jamais en clair

le transport

→ HTTPS, pas d'authentification sur du HTTP

l'entrée

→ taille bornée, schéma Pydantic, limitation de débit

les journaux

→ ni texte personnel, ni jeton, ni clé

une clé poussée sur GitHub est compromise, même effacée au commit suivant

Les secrets d'abord. Une clé dans le code part avec le dépôt et reste dans l'historique. Le `.env` du TP8 est ignoré par Git ; `.env.example` documente les noms sans les valeurs, et GitHub Actions lit ses *secrets*.

HTTPS n'est pas négociable dès qu'un mot de passe ou un jeton circule : sur HTTP, tout intermédiaire lit l'en-tête d'authentification tel quel. En pratique, un proxy devant les conteneurs du TP8.

La limitation de débit protège du coût, pas seulement de l'attaque. Un formulaire laissé ouvert et une boucle de dix lignes suffisent à saturer l'API, ou à faire grimper une facture d'inférence.

Le schéma Pydantic du TP3 est déjà une défense : il refuse ce qui n'a pas la bonne forme avant que le code métier ne s'exécute. Y ajouter une longueur maximale ferme l'envoi d'un texte de dix mégaoctets.

L'injection par les données, vue en semaine 10, vaut ici aussi : un document indexé au TP9 peut contenir des instructions. Les traiter comme du texte.

Semaine 11 · La page la plus utile est celle des limites

extrait attendu de docs/limites.md

Ce que le système ne sait pas faire

- juger l'ironie : « quel chef-d'œuvre... » est classé positif
- répondre à une question absente du corpus : il refuse, c'est voulu
- traiter une autre langue que le français
- garantir un délai sous charge : mesuré à un seul utilisateur
- expliquer une décision autrement que par les mots présents

chaque ligne renvoie au test ou à la mesure qui l'établit

Un système qui ne documente pas ses limites promet tout. L'utilisateur découvre alors les limites à ses dépens, et la confiance ne revient pas.

Vous en avez déjà la matière. Chaque semaine a produit un échec mesuré : le texte d'espaces du TP3, les quatre F1 identiques du TP7, l'image de l'interface plus lourde que celle de l'API au TP8, le passage littéral classé dernier au TP9. Ces échecs ne sont pas des brouillons à cacher, ce sont les entrées de cette page.

Une limite utile est vérifiable. « Le modèle est parfois imprécis » n'apprend rien. « Le modèle est entraîné sur 411 termes de critiques de films et ne transfère pas à un autre domaine » indique où ne pas l'employer.

Distinguer le défaut de la décision. Refuser hors corpus n'est pas une faiblesse : c'est un comportement choisi et testé.

Le jury lira cette page en premier. Elle montre que vous savez où se situe votre système, ce qu'aucune démonstration réussie ne prouve.

Semaine 11 · La soutenance : 20 minutes, 15 % de la note

grille sur 20, annoncée à l'avance

démonstration qui fonctionne		5 pts
choix techniques justifiés		5 pts
mesures et limites assumées		5 pts
réponses aux questions		3 pts
tenue du temps, 20 min		2 pts

une démonstration enregistrée est acceptée si le direct échoue : préparez-la

Le format : 12 minutes de présentation, 8 minutes de questions. Le dépassement coûte des points ; le jury coupe à 12.

Ce qui rapporte le plus n'est pas la fonctionnalité en plus, c'est la mesure. Un binôme qui montre son jeu d'or, son taux de sourçage et sa page de limites passe devant un binôme qui montre une interface plus jolie.

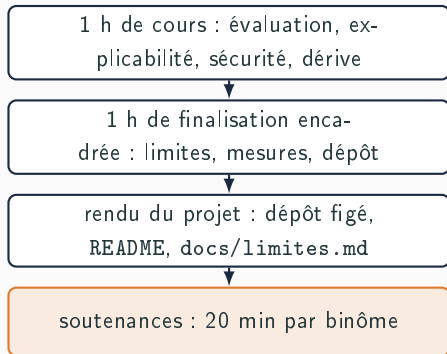
Trois questions posées à tout le monde : d'où vient ce chiffre ; que fait le système sur une entrée vide ou hors sujet ; qu'est-ce qui casserait à cent fois le volume.

Une erreur fréquente : dix minutes d'architecture et deux de démonstration. Inverser : le schéma sert à expliquer ce que le jury vient de voir marcher.

L'honnêteté est notée. « Ce cas n'est pas traité, il est dans limites.md » vaut mieux qu'une réponse vague.

Rappel du slide 05b : 30 % TP, 15 % étude de conception, 40 % projet, 15 % soutenance.

Semaine 11 · Le déroulé, et ce qui reste après



Kit de la semaine : `code/semaine11/` (SHAP, détection de dérive) et `semaine11-colab.ipynb`. Pas de TP11 : le cours en compte dix.

Le dépôt est figé au rendu. Ce qui est poussé après ne compte pas. Vérifiez la veille que le dépôt cloné sur une machine neuve s'installe et démarre : c'est exactement ce que fera le jury.

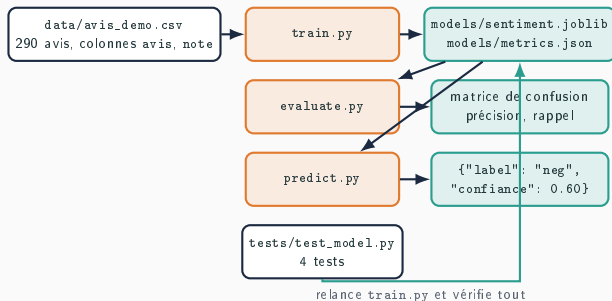
Ce que vous emportez tient en une phrase : un modèle n'est pas une application, et une démonstration n'est pas une preuve. Les onze semaines ont ajouté, à chaque étape, le moyen de le vérifier : un test, une mesure, un journal.

Les échecs conservés dans ce cours ne sont pas des accidents de préparation. Le texte d'espaces, les F1 ex æquo, le passage littéral classé dernier, la confiance qui monte pendant la dérive : ce sont les quatre façons dont un système ment sur son propre état.

Ce qui n'a pas été traité et que vous rencontrerez : le passage à l'échelle, l'entraînement distribué, la conformité réglementaire, l'annotation humaine à grande échelle. Les réflexes de mesure valent aussi là-bas.

L'annexe qui suit reprend le TP2 pas à pas, pour ceux qui veulent refaire le fil rouge depuis le début.

TP2 · Objectif : un modèle sauvegardé, mesuré et testé



Vous recevez : le kit `code/semaine2/` (quatre scripts, un générateur de données, quatre tests) et `TP2-colab.ipynb` pour travailler sans installation.

Vous rendez : un dépôt où `pytest` est vert, `models/metrics.json` calculé sur le vrai jeu Allociné, et `docs/tp2.md` avec le tableau comparatif de l'étape D.

Installation (dans le venv du TP1) :

```
pip install pandas scikit-learn joblib pytest
cd code/semaine2
python data/make_demo.py # 290 avis
```

Windows : mêmes commandes dans PowerShell ;

Colab : ouvrir le notebook et exécuter les cellules dans l'ordre.

TP2 · Étape A (20 min) : lire train.py avant de le lancer

```
df = pd.read_csv(CSV) # 1
X_tr, X_te, y_tr, y_te = train_test_split( # 2
    df[avis], df[note], test_size=0.2,
    random_state=42, stratify=df[note])
pipe = Pipeline([ # 3
    ("tfidf", TfidfVectorizer(ngram_range=(1, 2))),
    ("clf", LogisticRegression(max_iter=1000))])
pipe.fit(X_tr, y_tr) # 4
pred = pipe.predict(X_te) # 5
f1 = f1_score(y_te, pred, average="macro")
joblib.dump(pipe, "models/sentiment.joblib") # 6
json.dump({"f1_macro": round(f1, 3), ...},
    open("models/metrics.json", "w"))
```

```
$ python train.py
      precision    recall  f1-score   support
neg         1.000      0.897      0.945        29
pos         0.906      1.000      0.951        29

F1 macro (test) : 0.948
Modele sauvegarde : models/sentiment.joblib
```

1. Lecture du CSV : deux colonnes, avis (texte) et note (pos / neg).
2. Séparation 80 / 20 ; random_state rend le tirage reproductible, stratify garde la proportion des classes.
3. Un seul objet : vectorisation puis classifieur.
4. fit n'a vu que le jeu d'entraînement.
5. Les métriques sont calculées sur le jeu de test, jamais vu.
6. Deux fichiers sortent : le modèle et ses métriques.

À faire : lancer, noter le F1. Passer test_size à 0,5 : le F1 bouge-t-il ? Pourquoi ?

Question piège : que se passe-t-il si on remplace X_te par X_tr à l'étape 5 ?

TP2 · Étape B (20 min) : lire les erreurs, pas seulement le score

```
pipe = joblib.load("models/sentiment.joblib") # on recharge
pred = pipe.predict(X_te)                     # meme
decoupage
cm = confusion_matrix(y_te, pred, labels=["pos", "neg"])
```

réel \ prédit	pos	neg
pos	29	0
neg	3	26

```
# afficher les avis mal classes
erreurs = X_te[pred != y_te]
for a, r, p in zip(erreurs, y_te[pred != y_te], pred[pred !=
y_te]):
    print(f"réel {r} / prédit {p}] {a}")
```

Pourquoi recharger : le fichier .joblib doit suffire à lui seul, comme pour l'API en semaine 3.

Lire la matrice : ligne = vérité, colonne = prédiction. Les 3 erreurs sont des avis négatifs prédits positifs. Avec le bloc de code ci-contre, vous les affichez : ce sont les avis mitigés (« brillant au début, mais finalement une déception »).

À écrire dans docs/tp2.md : pour chaque erreur, le mot qui a trompé le modèle, et ce que TF-IDF ne voit pas (l'ordre, le « mais »).

Question : pour un service qui alerte sur les clients mécontents, quelle case de la matrice coûte le plus cher ?

TP2 · Étapes C et D (50 min) : le vrai jeu, puis comparer honnêtement

C. Charger Allociné (5 000 avis, Hugging Face) :

```
from datasets import load_dataset
ds = load_dataset("allocine", split="train[:5000]")
df = pd.DataFrame({"avis": ds["review"],
                  "note": ["pos" if 1 == 1 else "neg" for 1 in ds["label"]
                           ]})
df.to_csv("data/allocine_5000.csv", index=False)
```

```
$ python train.py data/allocine_5000.csv
$ python evaluate.py data/allocine_5000.csv
```

D. Comparer : une seule chose change à la fois.

Variante	Vectoriseur	Classifieur	F1 macro
référence	(1,2)	LogisticRegression	...
V1	(1,1)	LogisticRegression	...
V2	(1,2)	LinearSVC	...
V3	(1,2) + min_df=2	LogisticRegression	...

Fixer random_state=42 et le même découpage pour toutes les variantes



Changer un seul paramètre, relancer train.py, noter le F1



Écart inférieur à 0,01 : considérer les variantes équivalentes



Garder la plus simple à expliquer, pas la plus haute au centième

Attendu sur Allociné : F1 macro entre 0,85 et 0,92 selon la variante ; le jeu de démonstration (0,95) est plus facile que la réalité, c'est normal.

Piège : comparer deux variantes sur deux découpages différents. La différence mesurée serait celle du tirage, pas des modèles.

TP2 · Étape E (30 min) : tester, ignorer le modèle, livrer

```
@pytest.fixture(scope="module", autouse=True)
def modele_entraine():
    # entraine une fois pour tous
    les tests
    subprocess.run([sys.executable, "train.py"], cwd=RACINE)

def test_fi_minimal():
    m = json.load(open(RACINE / "models/metrics.json"))
    assert m["fi_macro"] >= 0.80

def test_predictions_coherentes():
    pipe = joblib.load(RACINE / "models/sentiment.joblib")
    assert pipe.predict(["Un film magnifique, j'ai adore."])
        [0] == "pos"
    assert pipe.predict(["Un film ennuyeux, a eviter."]) [0] ==
        "neg"
```

```
$ python -m pytest -q
.... [100%]
4 passed in 1.98s
$ cat .gitignore
.venv/ models/*.joblib data/allocine_5000.csv
```

Les 4 tests : fichiers présents; F1 macro $\geq 0,80$ (non-régression); deux avis de bon sens bien classés; `predict.py` renvoie label et confiance.

Pourquoi ignorer le .joblib : il se régénère en 2 s, pèse vite plusieurs Mo, et sera versionné par MLflow (S7). `metrics.json`, lui, est commité : c'est la trace du résultat.

Grille (sur 10) : tests verts en CI (3); `metrics.json` sur Allociné (2); tableau de l'étape D (3); analyse des erreurs (2).

Semaine 3 : `predict.py` est appelé par la route POST `/predict` de l'API FastAPI; gardez sa signature.